



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Τίτλος Πτυχιακής Εργασίας	Ανάπτυξη Εφαρμογής Quiz για Κινητές Συσκευές με χρήση .NET MAUI, ASP.NET Core και MySQL Development of a Quiz Application for Mobile Devices Using .NET MAUI, ASP.NET Core and MySQL
Ονοματεπώνυμο Φοιτητή	Μπούτσικος Γεώργιος Κωνσταντίνος
Πατρώνυμο	Ιωάννης
Αριθμός Μητρώου	Π20141
Επιβλέπων	Αλέπης Ευθύμιος, Καθηγητής

Ημερομηνία Παράδοσης: Ιούνιος 2025

Copyright ©

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν αποκλειστικά τον συγγραφέα και δεν αντιπροσωπεύουν τις επίσημες θέσεις του Πανεπιστημίου Πειραιώς.

Ως συγγραφέας της παρούσας εργασίας, δηλώνω πως δεν αποτελεί προϊόν λογοκλοπής και δεν περιέχει υλικό από μη αναφερόμενες πηγές.

Ευχαριστίες

Καταρχήν, θα ήθελα να ευχαριστήσω τον κ. Αλέπη για την βοήθεια του καθώς και τους καθηγητές του τμήματος που με καθοδήγησαν σε αυτό το ακαδημαϊκό ταξίδι, παρέχοντας μου τις απαραίτητες γνώσεις, σημαντικά εφόδια και την αγάπη για αυτή την επιστήμη που ονομάζουμε «Πληροφορική». Επίσης, θα ήθελα να ευχαριστήσω τους καθηγητές που με προετοίμασαν για την διαδικασία των Πανελληνίων αλλά και που μου δίδαξαν πολλά μαθήματα ζωής. Ακόμη, ευχαριστίες πρέπει να απευθυνθούν στο προσωπικό του Πανεπιστημίου και στην γραμματεία, που φροντίζουν να εξυπηρετούν εμάς τους φοιτητές καθημερινά. Τέλος, ένα μεγάλο ευχαριστώ στους συμφοιτητές & φίλους για την στήριξη τους, αλλά ειδικότερα στην οικογένεια μου, που με κόπο και θυσίες χρόνων έδωσαν πολύτιμο αγώνα για να μου προσφέρουν ένα καλύτερο μέλλον. Σας ευχαριστώ όλους από καρδιάς.

Περίληψη

Η παρούσα πτυχιακή εργασία παρουσιάζει την ανάπτυξη μιας εφαρμογής Quiz για κινητές συσκευές, η οποία συνδυάζει την ψυχαγωγία με τη μάθηση. Στόχος της εφαρμογής είναι η παροχή μιας διαδραστικής εμπειρίας στον χρήστη, μέσα από την απάντηση ερωτήσεων πολλαπλής επιλογής σε διάφορες θεματικές κατηγορίες, συνδυάζοντας ταυτόχρονα ποικίλες τεχνολογίες, τεχνικές και γνώσεις από πολλά αντικείμενα της πληροφορικής. Η εργασία περιλαμβάνει την πλήρη περιγραφή του σχεδιασμού, της αρχιτεκτονικής και των επιμέρους λειτουργιών της εφαρμογής, ενώ επισημαίνονται οι τεχνικές προκλήσεις και προτείνονται δυνατότητες μελλοντικής επέκτασης.

Abstract

This undergraduate thesis presents the development of a Quiz application for mobile devices, which combines entertainment with learning. The goal of the application is to provide users with interactive experience through answering multiple-choice questions across various thematic categories, while simultaneously integrating a range of technologies, techniques, and knowledge from multiple areas of computer science. The thesis includes a comprehensive description of the design, architecture, and individual functionalities of the application, highlights the technical challenges encountered, and proposes possibilities for future expansion.

Πίνακας Περιεχομένων

Copyright ©.....	1
Ευχαριστίες.....	2
Περίληψη	3
Abstract	3
Εισαγωγή.....	6
Κεφάλαιο 1: Αναλυτική Περιγραφή.....	7
1.1 Περιγραφή Εφαρμογής	7
1.2 Επιστημονικό Πλαίσιο	7
1.3 Ανάγκες που Καλύπτονται	7
Κεφάλαιο 2: Μεθοδολογική Προσέγγιση	8
2.1 Επιλογή Τεχνολογιών	8
2.1.1 .NET MAUI (Multi-platform App UI).....	8
2.1.2 ASP.NET Core Web API	8
2.1.3 Entity Framework Core & MySQL	8
2.1.4 JSON & HTTP Client.....	8
2.2 Αρχιτεκτονική Συστήματος	9
2.2.1 Ροή Λειτουργίας	9
2.2.2 Σχεδιαστικές Αποφάσεις	9
2.2.3 Δομή	10
Κεφάλαιο 3: Backend	13
3.1 API	13
3.1.1 Data Models	13
3.1.2 Database Context.....	15
3.1.3 Data Transfer Objects	15
3.1.4 Business Logic (Controllers)	16
3.1.5 Configuration	18
3.2 Βάση Δεδομένων	20
Κεφάλαιο 4: Frontend.....	21
4.1 Αρχική Σελίδα	21
4.2 Σύνδεση/Εγγραφή.....	24
4.3 Κατηγορίες.....	28
4.4 Παιχνίδι	29
4.5 Αποτελέσματα	35
4.6 Ρυθμίσεις.....	39
4.7 API Service.....	42
4.8 Audio Service	44
Κεφάλαιο 5: Αποτελέσματα	46

5.1 Δεδομένα και Παραδοχές	46
5.2 Παρουσίαση Αποτελεσμάτων	46
5.3 Σχολιασμός Αποτελεσμάτων	46
Κεφάλαιο 6: Προκλήσεις & Προβλήματα	47
6.1 Διαχείριση Επικοινωνίας Frontend – Backend	47
6.2 Υλοποίηση Leaderboards και Ενημέρωση Σκορ	47
6.3 Debugging	47
Κεφάλαιο 7: Προτεινόμενες Βελτιώσεις	48
7.1 Προσθήκη Ερωτήσεων μέσω Backend Panel	48
7.2 Achievements	48
7.3 Πολλαπλές Γλώσσες	48
7.4 Game Modes	48
Κεφάλαιο 8: Εγχειρίδιο Χρήστη	49
8.1 Αρχική Σελίδα	49
8.2 Σύνδεση/Εγγραφή	50
8.3 Επιλογή Κατηγορίας	50
8.4 Απάντηση Ερωτήσεων	51
8.5 Αποτελέσματα	51
8.6 Ρυθμίσεις	52
8.7 Αποσύνδεση/Εξοδος	52
Συμπεράσματα	53
Πίνακας Ορολογίας & Αρκτικόλεξων	54
Βιβλιογραφία	55

Εισαγωγή

Η σύγχρονη εποχή της ψηφιακής τεχνολογίας χαρακτηρίζεται από τη ραγδαία εξάπλωση των έξυπνων συσκευών και των εφαρμογών λογισμικού που αποσκοπούν όχι μόνο στην ενημέρωση και την επικοινωνία, αλλά και στην ψυχαγωγία και την εκπαίδευση. Οι εφαρμογές τύπου κουίζ (π.χ. “Πες Βρες”), συνδυάζουν την ψυχαγωγία με τη μάθηση, προσφέροντας έναν ευχάριστο και ταυτόχρονα παιδαγωγικό τρόπο απόκτησης γνώσεων. Έτσι λοιπόν, αντλώντας έμπνευση από τέτοιου είδους εφαρμογές, σε συνδυασμό με τις γνώσεις που αποκόμισα από τα παιδαγωγικά μαθήματα που προσφέρει το τμήμα, είχα την ιδέα να αναπτύξω κι εγώ ο ίδιος ένα τέτοιο μέσο μάθησης αλλά και διασκέδασης.

Το αντικείμενο αυτό παρουσιάζει ιδιαίτερο επιστημονικό ενδιαφέρον, καθώς συνδυάζει στοιχεία ανάπτυξης εφαρμογών πολλαπλών πλατφορμών (cross-platform development), δημιουργίας API & διαχείρισης βάσεων δεδομένων, ενώ παράλληλα επιτρέπει την ενσωμάτωση λειτουργιών gamification που αυξάνουν την εμπλοκή και την αλληλεπίδραση των χρηστών.

Σύμφωνα με τη διεθνή βιβλιογραφία, εφαρμογές αυτού του τύπου χρησιμοποιούνται ευρέως στην εκπαίδευση (εκπαιδευτικά κουίζ), στην ενδυνάμωση της μνήμης, στην ψυχαγωγία, αλλά και στη συλλογή δεδομένων για ερευνητικούς σκοπούς.

Ερευνητικές κατευθύνσεις εστιάζουν στη βελτιστοποίηση της εμπειρίας χρήστη, στη μηχανική μάθηση για την προσαρμογή της δυσκολίας των ερωτήσεων και στην κοινωνική διάσταση μέσω leaderboards και multiplayer λειτουργιών.

Η βασική στόχευση της παρούσας πτυχιακής είναι η κατασκευή μιας πλήρως λειτουργικής εφαρμογής quiz που υποστηρίζει:

- την εγγραφή και είσοδο χρηστών,
- την επιλογή θεματικής κατηγορίας ερωτήσεων,
- την αξιολόγηση των γνώσεων του χρήστη μέσα από κουίζ πολλαπλής επιλογής,
- την αποθήκευση και προβολή των υψηλότερων επιδόσεων (highscores) ανά κατηγορία μέσω δυναμικών leaderboards.

Οι επιμέρους στόχοι της εργασίας είναι οι εξής:

- Η δημιουργία εύχρηστου περιβάλλοντος χρήστη για κινητές συσκευές.
- Η κατασκευή ενός RESTful backend που διαχειρίζεται εγγραφές, συνδέσεις, σκορ και leaderboards.
- Η αποθήκευση δεδομένων με ασφάλεια
- Η δυναμική εμφάνιση των υψηλότερων σκορ ανά κατηγορία και χρήστη.

Η επιστημονική συνεισφορά της πτυχιακής εντοπίζεται στην ολοκληρωμένη προσέγγιση ανάπτυξης μίας εκπαιδευτικής/ψυχαγωγικής εφαρμογής, με δυνατότητες επέκτασης για μελλοντική αξιοποίηση.

Η εργασία διαρθρώνεται ως εξής:

- ✓ Κεφάλαιο 1: Αναλυτική Περιγραφή
- ✓ Κεφάλαιο 2: Μεθοδολογική Προσέγγιση
- ✓ Κεφάλαιο 3: Backend
- ✓ Κεφάλαιο 4: Frontend
- ✓ Κεφάλαιο 5: Αποτελέσματα
- ✓ Κεφάλαιο 6: Προκλήσεις & Προβλήματα
- ✓ Κεφάλαιο 7: Προτεινόμενες Βελτιώσεις
- ✓ Κεφάλαιο 8: Εγχειρίδιο Χρήστη

Κεφάλαιο 1: Αναλυτική Περιγραφή

1.1 Περιγραφή Εφαρμογής

Η παρούσα πτυχιακή εργασία αφορά τη σχεδίαση και υλοποίηση μιας εφαρμογής ερωτήσεων γνώσεων τύπου quiz, με χρήση της πλατφόρμας .NET MAUI για το frontend και ASP.NET Core Web API για το backend. Στόχος είναι η δημιουργία μιας πλήρως λειτουργικής, εύχρηστης και επεκτάσιμης εφαρμογής που επιτρέπει στους χρήστες να συμμετέχουν σε quiz γνώσεων, να καταγράφουν επιδόσεις και να ανταγωνίζονται μέσω δυναμικών πινάκων κατάταξης.

1.2 Επιστημονικό Πλαίσιο

Η ανάπτυξη εκπαιδευτικών ή ψυχαγωγικών εφαρμογών έχει αυξηθεί τα τελευταία χρόνια, κυρίως λόγω της δημοτικότητας που έχουν αποκτήσει τέτοιες εφαρμογές. Στη βιβλιογραφία, δίνεται έμφαση τόσο σε τεχνολογικά θέματα (όπως σχεδίαση RESTful APIs, αποθήκευση και ανάκτηση δεδομένων με χρήση ORM, συγχρονισμός μεταξύ client και server), όσο και σε παιδαγωγικά θέματα (π.χ. πώς δομείται ένα ερώτημα για να διατηρεί το ενδιαφέρον του χρήστη). Επιπλέον, σε επίπεδο βάσεων δεδομένων, εξετάζεται η αποδοτική αποθήκευση και ανάκτηση των αποτελεσμάτων των χρηστών με γνώμονα τη διατήρηση μόνο των απαραίτητων δεδομένων.

1.3 Ανάγκες που Καλύπτονται

Η εφαρμογή παρέχει στους χρήστες:

- Διαδραστικό περιβάλλον quiz με πολλαπλές κατηγορίες
- Δυναμική ενημέρωση highscore ανά χρήστη και κατηγορία
- Προσωποποιημένη εμπειρία μέσω avatar και συστήματος εισόδου/εγγραφής
- Προβολή πίνακα κατάταξης

Κεφάλαιο 2: Μεθοδολογική Προσέγγιση

2.1 Επιλογή Τεχνολογιών

Για το frontend χρησιμοποιήθηκε το .NET MAUI (Multi-platform App UI), το οποίο επιτρέπει την ανάπτυξη εφαρμογών για Android, iOS, Windows και macOS με κοινό κώδικα. Για το backend χρησιμοποιήθηκε ASP.NET Core Web API σε συνδυασμό με Entity Framework Core για την πρόσβαση σε βάση δεδομένων MySQL. Ο επιμέρους κώδικας και η εκτέλεση της εφαρμογής πραγματοποιήθηκαν στο Visual Studio.

2.1.1 .NET MAUI (Multi-platform App UI)

Το .NET MAUI αποτελεί την εξέλιξη του Xamarin και επιτρέπει την ανάπτυξη εφαρμογών με κοινό κώδικα για Android, iOS, Windows και macOS. Επιλέχθηκε λόγω της ευκολίας χρήσης του και της δυνατότητας γρήγορης ανάπτυξης UI.

2.1.2 ASP.NET Core Web API

Η backend υπηρεσία αναπτύχθηκε με ASP.NET Core Web API, το οποίο επιτρέπει τη δημιουργία RESTful endpoints για αποστολή και λήψη δεδομένων μεταξύ του client και της βάσης.

2.1.3 Entity Framework Core & MySQL

Το Entity Framework Core χρησιμοποιήθηκε ως ORM για την αλληλεπίδραση με τη βάση δεδομένων MySQL. Η MySQL είναι ιδανική για εφαρμογές, με δυνατότητα τοπικής αποθήκευσης χωρίς ανάγκη για server.

2.1.4 JSON & HTTP Client

Για την επικοινωνία με το Web API χρησιμοποιούνται JSON μορφή δεδομένων και HTTP Client, με χρήση των βιβλιοθηκών System.Net.Http.Json.

2.2 Αρχιτεκτονική Συστήματος

Η εφαρμογή βασίζεται σε μια client-server αρχιτεκτονική:

- ❖ **Client:** Το quiz app σε .NET MAUI, που λειτουργεί ως το γραφικό περιβάλλον του χρήστη
- ❖ **Server:** Το Web API που χειρίζεται αιτήματα εγγραφής, σύνδεσης, αποθήκευσης και ανάκτησης highscores
- ❖ **Database:** Μια βάση που περιέχει δύο βασικούς πίνακες: Users & Highscores

2.2.1 Ροή Λειτουργίας

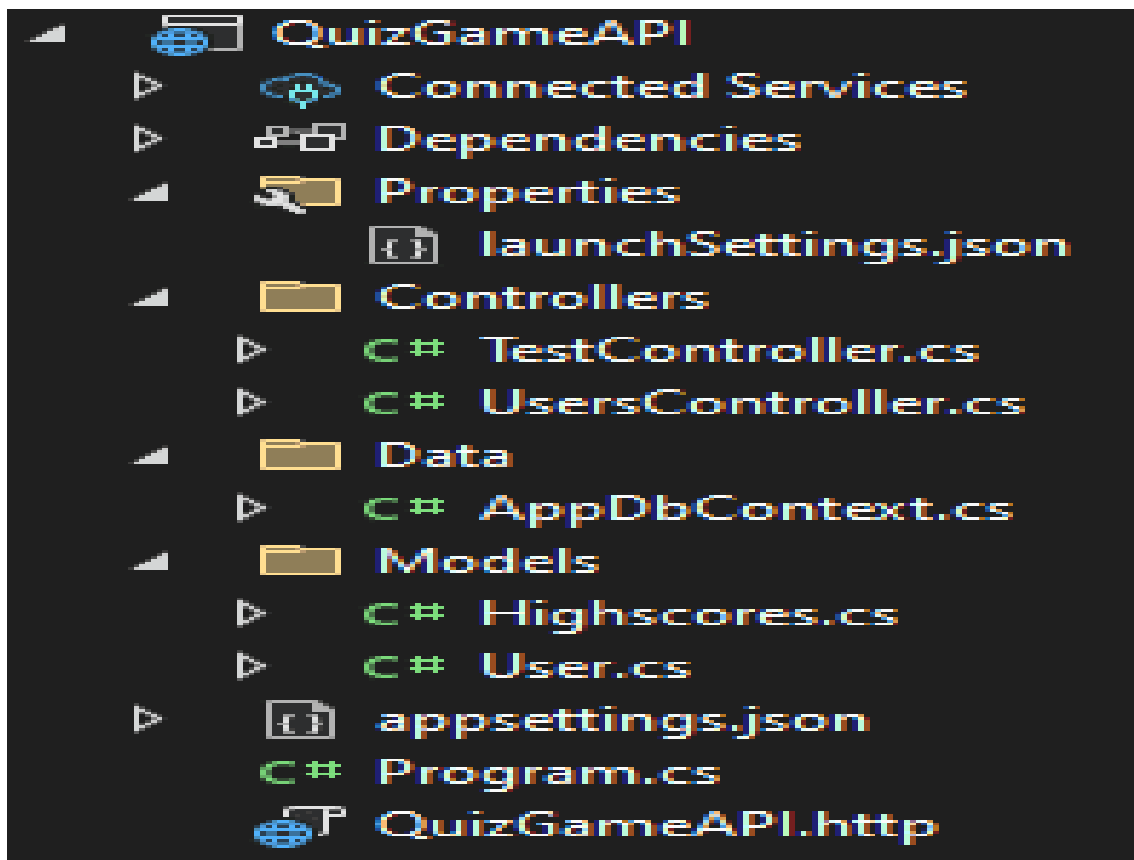
Ο χρήστης ακολουθεί την εξής ροή:

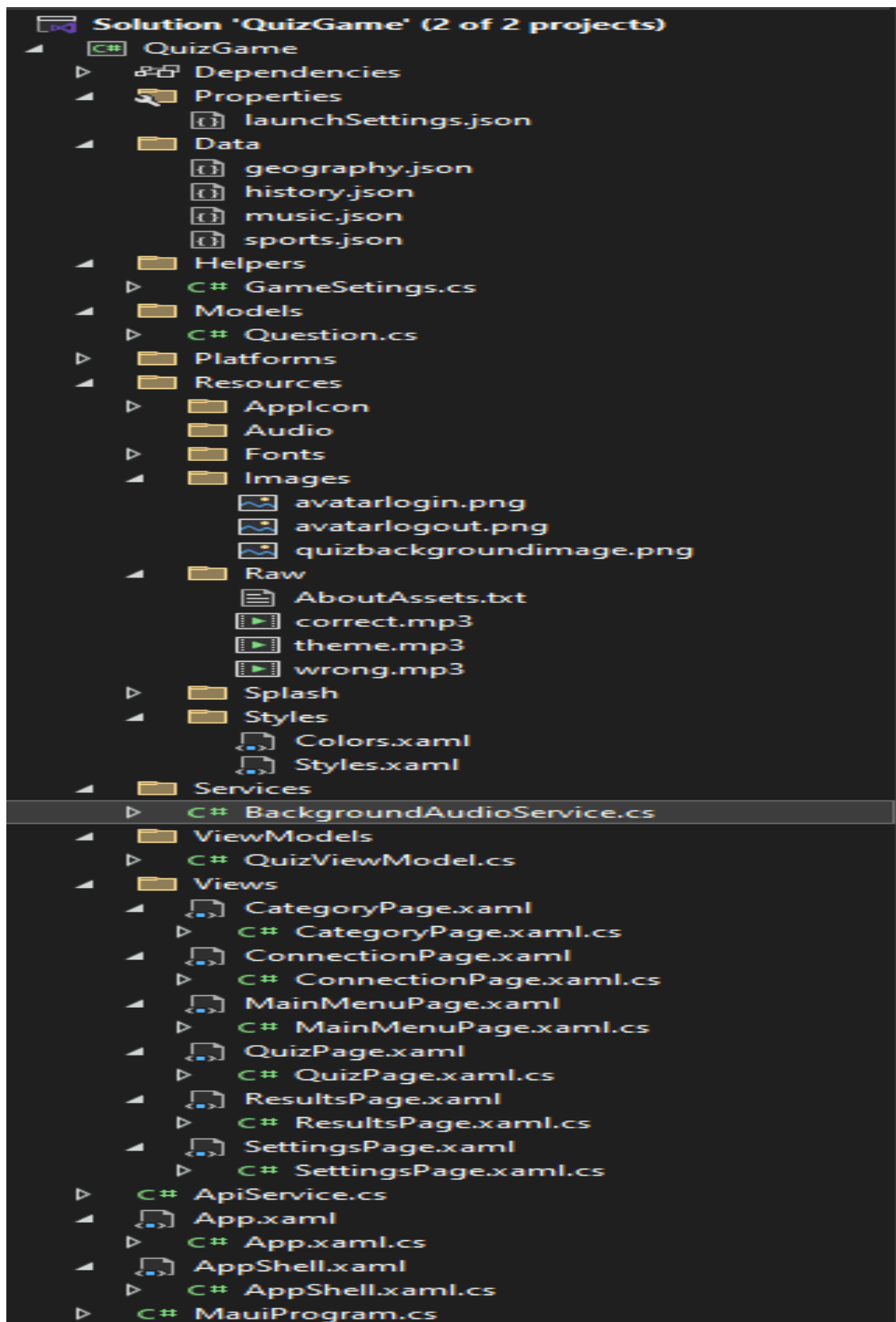
1. Πραγματοποιεί εγγραφή ή σύνδεση.
2. Επιλέγει κατηγορία quiz.
3. Παίζει το quiz και υπολογίζεται το σκορ.
4. Ο χρήστης βλέπει την κατάταξή του στα leaderboards.

2.2.2 Σχεδιαστικές Αποφάσεις

Διατηρείται μόνο το υψηλότερο σκορ κάθε χρήστη ανά κατηγορία, για μείωση του μεγέθους των δεδομένων. Ο χρήστης αναγνωρίζεται με βάση το username που αποθηκεύεται στην ΒΔ. Η λειτουργία leaderboards βασίζεται σε δυναμικά queries της βάσης, με ταξινόμηση κατά σκορ.

2.2.3 Δομή





Κεφάλαιο 3: Backend

Προκειμένου να πραγματοποιηθεί η διαχείριση και επεξεργασία των δεδομένων, όπως και των λειτουργιών που εκτελούνται στην εφαρμογή, έχει υλοποιηθεί ένα background service, το οποίο επικοινωνεί με την κύρια εφαρμογή καθώς και με την βάση δεδομένων.

3.1 API

Ένα API (Application Programming Interface) είναι ένα σύνολο κανόνων και μεθόδων που επιτρέπουν σε διαφορετικές εφαρμογές να επικοινωνούν και να ανταλλάσσουν δεδομένα μεταξύ τους. Αρχικά, η κύρια εφαρμογή στέλνει ένα HTTP αίτημα στο API. Έπειτα, το τελευταίο ελέγχει/επεξεργάζεται την ΒΔ και στέλνει πίσω μια HTTP απάντηση με το αποτέλεσμα της ενέργειας. Με αυτόν τον τρόπο επιτυγχάνεται η επικοινωνία μεταξύ frontend & backend, η καλύτερη οργάνωση και καταμερισμός εργασιών & η ανεξαρτησία μεταξύ των εφαρμογών.

3.1.1 Data Models

Αντιπροσωπεύουν τις οντότητες της εφαρμογής και αντιστοιχούν στους πίνακες της βάσης. Επομένως, χρειάζονται δύο, ένα για τους χρήστες & ένα για τα σκορ.

```
using System.ComponentModel.DataAnnotations.Schema;

namespace QuizGameAPI.Models
{
    [Table("users")]
    3 references
    public class User
    {
        [Column("id")]
        3 references
        public int id { get; set; }

        [Column("username")]
        6 references
        public string username { get; set; }

        [Column("password_hash")]
        2 references
        public string password_hash { get; set; }

        0 references
        public ICollection<Highscores> Highscores { get; set; }
    }
}
```

```
using System.ComponentModel.DataAnnotations.Schema;

namespace QuizGameAPI.Models
{
    [Table("highscores")]
    3 references
    public class Highscores
    {
        0 references
        public int Id { get; set; }

        2 references
        public int UserId { get; set; }

        3 references
        public string Category { get; set; }

        5 references
        public int Highscore { get; set; }

        1 reference
        public User User { get; set; }
    }
}
```

3.1.2 Database Context

Η AppDbContext κλάση λειτουργεί ως γέφυρα ανάμεσα στον κώδικα και τη βάση. Περιλαμβάνει DbSet<User> Users κ.λπ., και χρησιμοποιείται από το Entity Framework για να εκτελεί ερωτήματα.

```
using Microsoft.EntityFrameworkCore;
using QuizGameAPI.Models;

7 references
public class AppDbContext : DbContext
{
    0 references
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    6 references
    public DbSet<User> Users { get; set; }

    4 references
    public DbSet<Highscores> Highscores { get; set; }
}
```

3.1.3 Data Transfer Objects

Είναι "σχήματα" που χρησιμοποιούνται για να σταλθούν/ληφθούν δεδομένα μέσω των endpoints, αντί να εκτεθούν ολόκληρα τα μοντέλα.

```
2 references
public class UserDto
{
    5 references
    public string Username { get; set; }
    4 references
    public string Password { get; set; }
}

1 reference
public class UpdateCategoryHighscoreDto
{
    1 reference
    public string Username { get; set; }
    2 references
    public string Category { get; set; }
    3 references
    public int Score { get; set; }
}
```


3.1.4 Business Logic (Controllers)

Είναι η λογική της εφαρμογής που υλοποιείται μέσα στους controllers ή services. Ο UsersController αποτελεί τον βασικό controller του backend API της εφαρμογής και είναι υπεύθυνος για τη διαχείριση χρηστών καθώς και την καταγραφή και ανάκτηση βαθμολογιών. Ο controller εκθέτει διάφορα HTTP endpoints που μπορούν να προσπελαστούν από το frontend μέσω RESTful αιτημάτων. Παρακάτω ακολουθεί η περιγραφή των λειτουργιών του:

1. Καταχώρηση νέου χρήστη (Register)

Δέχεται ένα JSON αντικείμενο τύπου UserDto που περιέχει username και password. Ελέγχει αν το όνομα χρήστη υπάρχει ήδη στη βάση. Αν όχι, δημιουργεί νέο χρήστη αποθηκεύοντας τον κωδικό με hash για ασφάλεια (BCrypt). Επιστρέφει OK αν η εγγραφή είναι επιτυχής.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using QuizGameAPI.Models;

namespace QuizGameAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class UsersController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public UsersController(ApplicationDbContext context)
        {
            _context = context;
        }

        [HttpPost("register")] // REGISTER
        public async Task<IActionResult> Register(UserDto userDto)
        {
            if (string.IsNullOrWhiteSpace(userDto.Username) || string.IsNullOrWhiteSpace(userDto.Password))
                return BadRequest("Username and password are required.");

            var exists = await _context.Users.AnyAsync(u => u.username == userDto.Username);
            if (exists) return BadRequest("Username already taken.");

            var user = new User
            {
                username = userDto.Username,
                password_hash = BCrypt.Net.BCrypt.HashPassword(userDto.Password)
            };

            _context.Users.Add(user);
            await _context.SaveChangesAsync();

            return Ok("User created.");
        }
    }
}
```

2. Είσοδος χρήστη (Login)

Δέχεται username και password και επαληθεύει τα στοιχεία. Ελέγχει αν ο χρήστης υπάρχει και αν ο κωδικός είναι σωστός (με έλεγχο hash). Επιστρέφει τα βασικά στοιχεία του χρήστη (π.χ. id, username) ή σφάλμα σε περίπτωση αποτυχίας.

```
[HttpPost("login")] // LOGIN
0 references
public async Task<IActionResult> Login(UserDto userDto)
{
    if (string.IsNullOrWhiteSpace(userDto.Username) || string.IsNullOrWhiteSpace(userDto.Password))
        return BadRequest("Username and password are required.");

    var user = await _context.Users.FirstOrDefaultAsync(u => u.username == userDto.Username);
    if (user == null || !BCrypt.Net.BCrypt.Verify(userDto.Password, user.password_hash))
        return Unauthorized("Invalid credentials.");

    return Ok(new { user.id, user.username });
}
```

3. Έλεγχος σύνδεσης με τη βάση (Test)

Endpoint για δοκιμή της σύνδεσης με τη βάση δεδομένων. Επιστρέφει τον αριθμό των χρηστών ή μήνυμα σφάλματος.

```
[HttpGet("test")] // TEST
0 references
public async Task<IActionResult> TestConnection()
{
    try
    {
        var users = await _context.Users.ToListAsync();
        return Ok(new { success = true, message = $"Connected! Found {users.Count} users." });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { success = false, message = "Failed to connect to the database.", error = ex.Message });
    }
}
```

4. Ενημέρωση υψηλής βαθμολογίας ανά κατηγορία (UpdateCategoryHighscore)

Ελέγχει αν υπάρχει ήδη βαθμολογία για το συγκεκριμένο χρήστη και κατηγορία. Αν όχι, δημιουργεί νέα εγγραφή. Αν υπάρχει και η νέα βαθμολογία είναι μεγαλύτερη, την ενημερώνει. Εξασφαλίζει πως μόνο η υψηλότερη βαθμολογία αποθηκεύεται για κάθε χρήστη ανά κατηγορία.

```
[HttpPost("update-category-highscore")] // UPDATE HIGHSCORE FOR CATEGORY
0 references
public async Task<IActionResult> UpdateCategoryHighscore(UpdateCategoryHighscoreDto dto)
{
    var user = await _context.Users.FirstOrDefaultAsync(u => u.username == dto.Username);
    if (user == null) return NotFound("User not found.");

    var highscore = await _context.Highscores
        .FirstOrDefaultAsync(h => h.UserId == user.id && h.Category == dto.Category);

    if (highscore == null)
    {
        _context.Highscores.Add(new Highscores
        {
            UserId = user.id,
            Category = dto.Category,
            Highscore = dto.Score
        });
    }
    else if (dto.Score > highscore.Highscore)
    {
        highscore.Highscore = dto.Score;
        _context.Highscores.Update(highscore);
    }

    await _context.SaveChangesAsync();
    return Ok("Highscore recorded.");
}
```

5. Επιστροφή leaderboard (Top 10) ανά κατηγορία (GetLeaderboard)

Επιστρέφει τη λίστα των 10 χρηστών με την υψηλότερη βαθμολογία για τη συγκεκριμένη κατηγορία. Οι εγγραφές επιστρέφονται ταξινομημένες φθίνουσα με βάση τη βαθμολογία.

```
[HttpGet("leaderboard/{category}")] // LEADERBOARD
0 references
public async Task<IActionResult> GetLeaderboard(string category)
{
    var topScores = await _context.Highscores
        .Where(h => h.Category == category)
        .OrderByDescending(h => h.Highscore)
        .Take(10)
        .Select(h => new
        {
            h.User.username,
            h.Highscore
        })
        .ToListAsync();

    return Ok(topScores);
}
```

3.1.5 Configuration

Το “appsettings.json” είναι το κύριο αρχείο ρυθμίσεων παραμέτρων της ASP.NET Core εφαρμογής. Το “launchSettings.json” καθορίζει πώς ξεκινάει η εφαρμογή.

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:8024",
      "sslPort": 44356
    }
  },
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5165",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7192;http://localhost:5165",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=quizgame;User=root;Password=giorgos2002!;"
  }
}
```

3.2 Βάση Δεδομένων

Η βάση δεδομένων αποτελεί ένα βασικό στοιχείο της εφαρμογής, καθώς αποθηκεύει όλα τα δεδομένα που απαιτούνται για τη λειτουργία της, όπως οι χρήστες, οι κατηγορίες και τα σκορ (highscores). Στην υλοποίηση αυτή, χρησιμοποιείται μια σχεσιακή βάση δεδομένων η οποία επιτρέπει την οργανωμένη αποθήκευση και ανάκτηση των δεδομένων μέσω πινάκων με προκαθορισμένα πεδία. Η επικοινωνία μεταξύ της εφαρμογής και της βάσης δεδομένων γίνεται μέσω του Entity Framework Core, ενός ORM που επιτρέπει στον κώδικα να χειρίζεται τις εγγραφές της βάσης ως αντικείμενα, αποφεύγοντας τη χρήση απευθείας SQL queries. Με αυτόν τον τρόπο, η βάση δεδομένων υποστηρίζει τις βασικές λειτουργίες της εφαρμογής, όπως:

- Την καταγραφή και επικύρωση χρηστών (registration & login)
- Την αποθήκευση και ενημέρωση των προσωπικών highscore των χρηστών ανά κατηγορία
- Την ανάκτηση των δεδομένων για την εμφάνιση του leaderboard

Η χρήση της βάσης δεδομένων εξασφαλίζει την αξιοπιστία, την ασφάλεια και την αποδοτικότητα της εφαρμογής.

	Id	UserId	Category	Highscore
▶	1	7	geography	10
	2	7	music	4
	3	7	history	9
	5	8	history	7
	6	9	history	3

	id	username	password_hash
▶	1	george	\$2a\$11\$J1aF0zSg5E/8G0Zqk0n1deUeJ9gHoQgKz2c/5Mi3ffDUwRjJpq8ie
	7	a	\$2a\$11\$6.HuAFPzk.p/vsGzj1jBPuRBNSkOTjCr7nN05oM8KCCAzgCkzFPC
	8	b	\$2a\$11\$b0wPlm/GKr7KK6RCa.JnZOxom11sSc1JR/5TvpBUmsOkDFSe.fZcC
	9	c	\$2a\$11\$FB2KsHG16E6tDFe5vdqScOdCSgsjkhFTNxlto50F9fb2Ed237XBe

Κεφάλαιο 4: Frontend

Το frontend της εφαρμογής αποτελεί το κομμάτι που αλληλεπιδρά άμεσα με τον τελικό χρήστη, παρέχοντας τη διεπαφή και τα οπτικά στοιχεία για την πλοήγηση και τη χρήση των λειτουργιών της. Μέσω του frontend, ο χρήστης μπορεί να εγγραφεί και να συνδεθεί, να επιλέξει κατηγορίες κουίζ, να απαντήσει σε ερωτήσεις, και να δει τα αποτελέσματα και το leaderboard. Η σχεδίαση του frontend στοχεύει στην απλότητα, τη φιλικότητα προς τον χρήστη, καθώς και στην ομαλή και γρήγορη απόκριση, ώστε η εμπειρία χρήσης να είναι άνετη και ευχάριστη.

4.1 Αρχική Σελίδα

Η σελίδα MainMenuPage της εφαρμογής, αποτελεί το βασικό μενού του quiz. Η κλάση κληρονομεί από ContentPage, δηλαδή είναι μία οθόνη UI, και βρίσκεται στον χώρο QuizGame.Views.

Στον constructor της κλάσης, αρχικοποιούνται τα στοιχεία της σελίδας μέσω της μεθόδου InitializeComponent() και καλείται η μέθοδος UpdateLoginStatus(), η οποία ελέγχει αν υπάρχει αποθηκευμένο όνομα χρήστη (username) στο SecureStorage. Αν υπάρχει, εμφανίζεται ένα avatar που δείχνει ότι ο χρήστης είναι συνδεδεμένος, το μήνυμα καλωσορίσματος ενημερώνεται με το username, και το κουμπί αποσύνδεσης καθίσταται ορατό. Αντίθετα, αν δεν υπάρχει αποθηκευμένο username, εμφανίζεται διαφορετικό avatar, μήνυμα που δηλώνει ότι ο χρήστης δεν είναι συνδεδεμένος, και το κουμπί αποσύνδεσης κρύβεται.

Η μέθοδος OnPlayClicked εκτελείται όταν ο χρήστης πατήσει το κουμπί "Play" και τον μεταφέρει στη σελίδα MainPage, όπου ξεκινά το quiz. Η μέθοδος OnLoginSignupClicked εκτελείται όταν ο χρήστης πατήσει "Login/Signup" και τον οδηγεί στη σελίδα σύνδεσης (LoginPage). Αντίστοιχα, η OnSettingsClicked οδηγεί τον χρήστη στη σελίδα ρυθμίσεων (SettingsPage).

Η μέθοδος OnLogoutClicked εκτελείται όταν ο χρήστης πατήσει "Logout". Διαγράφει από το SecureStorage τα αποθηκευμένα στοιχεία σύνδεσης (username και password), καλεί ξανά την UpdateLoginStatus() για να ενημερώσει την οθόνη, και μετά από 1 δευτερόλεπτο μεταφέρει τον χρήστη πίσω στη σελίδα σύνδεσης.

Τέλος, η OnExitClicked είναι υπεύθυνη για τον τερματισμό της εφαρμογής. Ανάλογα με την πλατφόρμα, καλεί τις αντίστοιχες εντολές για να σταματήσει η διεργασία της εφαρμογής.

Συνολικά, η MainMenuPage αποτελεί το κεντρικό σημείο πλοήγησης της εφαρμογής, δίνοντας στον χρήστη τη δυνατότητα να παίξει το παιχνίδι, να συνδεθεί ή να αποσυνδεθεί, να δει τις ρυθμίσεις, και να τερματίσει την εφαρμογή, ενώ παράλληλα προσαρμόζει τη συμπεριφορά της ανάλογα με την κατάσταση σύνδεσης του χρήστη.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="QuizGame.Views.MainMenuPage"
  Title="Quiz Game">

  <Grid>
    <!-- Background image -->
    <Image Source="quizbackgroundimage.png"
      Aspect="AspectFill"/>

    <!-- Buttons -->
    <VerticalStackLayout Padding="30"
      Spacing="20"
      VerticalOptions="Center">

      <Label Text="Quiz Game"
        FontSize="32"
        HorizontalOptions="Center" />

      <Image x:Name="AvatarImage"
        HeightRequest="80"
        WidthRequest="80"
        Margin="0,10"
        Aspect="AspectFit"
        HorizontalOptions="Center" />

      <Label x:Name="WelcomeLabel"
        FontSize="16"
        HorizontalOptions="Center"
        TextColor="Gray" />

      <Button Text="Play" Clicked="OnPlayClicked"/>
      <Button Text="Settings" Clicked="OnSettingsClicked"/>
      <Button x:Name="LoginButton" Text="Login / Signup" Clicked="OnLoginSignupClicked"/>
      <Button x:Name="LogoutButton"
        Text="Log Out"
        Clicked="OnLogoutClicked"
        IsVisible="False"
        BackgroundColor="DarkRed"
        TextColor="White" />
      <Button Text="X Exit" Clicked="OnExitClicked"/>
    </VerticalStackLayout>
  </Grid>
</ContentPage>
```

```

namespace QuizGame.Views;

16 references
public partial class MainMenuPage : ContentPage
{
    2 references
    public MainMenuPage()
    {
        InitializeComponent();
        UpdateLoginStatus();
    }

    // PLAY QUIZ
    0 references
    private async void OnPlayClicked(object sender, EventArgs e)
    {
        await Navigation.PushAsync(new MainPage());
    }

    //UPDATE ACCOUNT STATUS
    2 references
    private async void UpdateLoginStatus()
    {
        var username = await SecureStorage.GetAsync("username");

        if (!string.IsNullOrEmpty(username))
        {
            AvatarImage.Source = "avatarlogin.png";
            WelcomeLabel.Text = $"Logged in as: {username}";
            LogoutButton.IsVisible = true;
        }
        else
        {
            AvatarImage.Source = "avatarlogout.png";
            WelcomeLabel.Text = "Not logged in.";
            LogoutButton.IsVisible = false;
        }
    }

    // LOGOUT
    0 references
    private async void OnLogoutClicked(object sender, EventArgs e)
    {
        SecureStorage.Remove("username");
        SecureStorage.Remove("password");

        UpdateLoginStatus();

        await Task.Delay(1000);
        await Navigation.PushAsync(new LoginPage());
    }

    // CONNECTION
    0 references
    private async void OnLoginSignupClicked(object sender, EventArgs e)
    {
        await Navigation.PushAsync(new LoginPage());
    }

    // SETTINGS
    0 references
    private async void OnSettingsClicked(object sender, EventArgs e)
    {
        await Navigation.PushAsync(new SettingsPage());
    }

    // EXIT
    0 references
    private void OnExitClicked(object sender, EventArgs e)
    {
        #if ANDROID
            Android.OS.Process.KillProcess(Android.OS.Process.MyPid());
        #elif WINDOWS
            System.Diagnostics.Process.GetCurrentProcess().Kill();
        #endif
    }
}

```


4.2 Σύνδεση/Εγγραφή

Η σελίδα LoginPage της εφαρμογής, επιτρέπει στον χρήστη να πραγματοποιήσει είσοδο (login) ή εγγραφή (sign up). Η κλάση LoginPage κληρονομεί από την ContentPage.

Ο constructor LoginPage() αρχικοποιεί τη σελίδα με τη μέθοδο InitializeComponent(), η οποία φορτώνει τα στοιχεία της διεπαφής χρήστη από το αντίστοιχο XAML αρχείο.

Η μέθοδος OnSignUpClicked ενεργοποιείται όταν ο χρήστης πατήσει το κουμπί εγγραφής. Πρώτα διαβάζει τα στοιχεία από τα πεδία εισαγωγής UsernameEntry και PasswordEntry, αφαιρώντας κενά διαστήματα από το όνομα χρήστη. Αν κάποιο από τα δύο πεδία είναι κενό ή περιέχει μόνο κενά, εμφανίζεται μήνυμα λάθους με κόκκινο χρώμα που ζητά από τον χρήστη να συμπληρώσει και τα δύο πεδία. Αν τα πεδία είναι συμπληρωμένα, δημιουργείται ένα αντικείμενο ApiService και καλείται η μέθοδος RegisterAsync, η οποία επικοινωνεί με το backend API για να καταχωρίσει τον νέο λογαριασμό. Αν η εγγραφή είναι επιτυχής, εμφανίζεται θετικό μήνυμα με πράσινο χρώμα και, μετά από μια σύντομη παύση, η σελίδα κλείνει (επιστρέφοντας στην προηγούμενη). Αν η εγγραφή αποτύχει (π.χ. επειδή το όνομα χρήστη είναι ήδη κατοχυρωμένο), εμφανίζεται κόκκινο μήνυμα αποτυχίας.

Η μέθοδος OnLoginClicked ενεργοποιείται όταν ο χρήστης πατήσει το κουμπί σύνδεσης. Δημιουργεί ένα αντικείμενο ApiService και καλεί τη μέθοδο LoginAsync, στέλνοντας τα στοιχεία σύνδεσης στο backend API για έλεγχο. Αν η σύνδεση είναι επιτυχής, εμφανίζεται μήνυμα επιτυχίας, ο χρήστης ειδοποιείται με ένα DisplayAlert, και μετά από μια σύντομη παύση, μεταφέρεται στη σελίδα του κύριου μενού (MainMenuPage). Ταυτόχρονα, το όνομα χρήστη αποθηκεύεται με ασφάλεια μέσω της SecureStorage.SetAsync, ώστε να μπορεί να ανακτηθεί αργότερα (π.χ. για εμφάνιση στο κύριο μενού ή για αυτόματη σύνδεση). Αν η σύνδεση αποτύχει, εμφανίζεται μήνυμα λάθους και ειδοποίηση μέσω DisplayAlert.

Συνοψίζοντας, ο κώδικας αυτός διαχειρίζεται τη διαδικασία σύνδεσης και εγγραφής ενός χρήστη στο σύστημα του παιχνιδιού, προσφέροντας αντίστοιχη οπτική ανατροφοδότηση και συνδέοντας το UI με το backend μέσω της υπηρεσίας ApiService.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="QuizGame.Views.LoginPage"
  Title="Login / Sign Up">

  <VerticalStackLayout Padding="30" Spacing="20" VerticalOptions="Center">

    <Entry x:Name="UsernameEntry" Placeholder="Username" />
    <Entry x:Name="PasswordEntry" Placeholder="Password" IsPassword="True" />

    <Button Text="Sign Up" Clicked="OnSignUpClicked" />
    <Button Text="Login" Clicked="OnLoginClicked" />

    <Label x:Name="MessageLabel" TextColor="Red" IsVisible="False" />

  </VerticalStackLayout>
</ContentPage>
```

```

namespace QuizGame.Views
{
    16 references
    public partial class LoginPage : ContentPage
    {
        2 references
        public LoginPage()
        {
            InitializeComponent();
        }

        // SIGNUP
        0 references
        private async void OnSignUpClicked(object sender, EventArgs e)
        {
            string username = UsernameEntry.Text?.Trim();
            string password = PasswordEntry.Text;

            if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
            {
                MessageLabel.Text = "Please enter a username and password.";
                MessageLabel.TextColor = Colors.Red;
                MessageLabel.IsVisible = true;
                return;
            }

            var apiService = new ApiService();
            var success = await apiService.RegisterAsync(username, password);

            if (success)
            {
                MessageLabel.TextColor = Colors.Green;
                MessageLabel.Text = "Account created successfully!";
                MessageLabel.IsVisible = true;
                await Task.Delay(1500);
                await Navigation.PopAsync();
            }
            else
            {
                MessageLabel.TextColor = Colors.Red;
                MessageLabel.Text = "Registration failed. Username may be taken.";
                MessageLabel.IsVisible = true;
            }
        }

        // LOGIN
        0 references
        private async void OnLoginClicked(object sender, EventArgs e)
        {
            var apiService = new ApiService();
            bool loggedIn = await apiService.LoginAsync(UsernameEntry.Text, PasswordEntry.Text);

            if (loggedIn)
            {
                await DisplayAlert("Success", "You are logged in!", "OK");
                MessageLabel.TextColor = Colors.Green;
                MessageLabel.Text = "Login successful!";
                MessageLabel.IsVisible = true;

                await Task.Delay(1000);
                await Navigation.PushAsync(new MainMenuPage());
                await SecureStorage.SetAsync("username", UsernameEntry.Text);
            }
            else
            {
                await DisplayAlert("Error", "Login failed!", "Try again");
                MessageLabel.TextColor = Colors.Red;
                MessageLabel.Text = "Invalid username or password.";
                MessageLabel.IsVisible = true;
            }
        }
    }
}

```


4.3 Κατηγορίες

Η σελίδα `CategoryPage` επιτρέπει στον χρήστη να επιλέξει μια κατηγορία ερωτήσεων πριν ξεκινήσει το παιχνίδι.

Η κύρια λειτουργία της σελίδας υλοποιείται μέσω της μεθόδου `OnCategoryClicked`, η οποία ενεργοποιείται όταν ο χρήστης πατήσει ένα κουμπί που αντιστοιχεί σε κάποια κατηγορία (όπως "History", "Sports", "Music" κ.λπ.). Η μέθοδος ανακτά το `CommandParameter` του κουμπιού που πατήθηκε — μια τιμή που περιέχει το όνομα της επιλεγμένης κατηγορίας — και το αποθηκεύει στη μεταβλητή `category`. Στη συνέχεια, καλεί την `Navigation.PushAsync(new QuizPage(category))`, για να μεταφέρει τον χρήστη στην επόμενη σελίδα (`QuizPage`), περνώντας ως όρισμα την επιλεγμένη κατηγορία, ώστε το quiz που θα εμφανιστεί να αφορά τη σωστή θεματική ενότητα.

```
using QuizGame.Views;

namespace QuizGame
{
    15 references
    public partial class CategoryPage : ContentPage
    {
        1 reference
        public CategoryPage ()
        {
            InitializeComponent();
        }

        // CATEGORY
        0 references
        private async void OnCategoryClicked(object sender, EventArgs e)
        {
            var button = (Button)sender;
            var category = button.CommandParameter.ToString();

            await Navigation.PushAsync(new QuizPage(category));
        }
    }
}
```

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="QuizGame.CategoryPage"
    Title="Quiz Game">

    <Grid>

        <!-- Background image -->
        <Image Source="quizbackgroundimage.png"
            Aspect="AspectFill"/>

        <!-- Buttons -->
        <VerticalStackLayout Padding="20" Spacing="15">

            <Label Text="Choose a Category"
                FontSize="24"
                HorizontalOptions="Center"
                Margin="0,40,0,20"/>

            <Button Text="History" Clicked="OnCategoryClicked" CommandParameter="history"/>
            <Button Text="Geography" Clicked="OnCategoryClicked" CommandParameter="geography"/>
            <Button Text="Music" Clicked="OnCategoryClicked" CommandParameter="music"/>
            <Button Text="Sports" Clicked="OnCategoryClicked" CommandParameter="sports"/>

        </VerticalStackLayout>

    </Grid>
</ContentPage>
```

4.4 Παιχνίδι

Η σελίδα QuizPage είναι υπεύθυνη για την εμφάνιση και διαχείριση του ίδιου του quiz. Η σελίδα δέχεται ως είσοδο την κατηγορία που επέλεξε ο χρήστης στην προηγούμενη οθόνη και προβάλλει μία-μία τις ερωτήσεις που αντιστοιχούν σε αυτή την κατηγορία. Ενσωματώνει λειτουργίες όπως χρονομέτρηση, επιλογή απάντησης, ανατροφοδότηση με χρώματα και ήχο, βαθμολογία και εμφάνιση αποτελεσμάτων.

Κατά την αρχικοποίηση της σελίδας μέσω του constructor QuizPage(string category), αποθηκεύεται η επιλεγμένη κατηγορία στην ιδιωτική μεταβλητή _category, φορτώνονται οι ερωτήσεις που ανήκουν σε αυτήν την κατηγορία μέσω της LoadCategoryQuestions(), και παράλληλα φορτώνονται τα ηχητικά αρχεία επιτυχίας και αποτυχίας μέσω της LoadSoundsAsync().

Η μέθοδος LoadCategoryQuestions διαβάζει ασύγχρονα τις ερωτήσεις από ένα αρχείο JSON που αντιστοιχεί στην κατηγορία, και στη συνέχεια καλεί τη ShowNextQuestion() για να εμφανίσει την πρώτη ερώτηση. Η εμφάνιση κάθε ερώτησης περιλαμβάνει την ερώτηση και τις απαντήσεις ως κουμπιά. Κάθε κουμπί αντιστοιχεί σε μία επιλογή και περιέχει ως CommandParameter τον δείκτη της απάντησης στη λίστα. Όταν ο χρήστης επιλέξει μία απάντηση, καλείται η μέθοδος OnOptionClicked.

Η μέθοδος OnOptionClicked ελέγχει αν η απάντηση που επέλεξε ο χρήστης είναι σωστή ή λάθος, συγκρίνοντας τον δείκτη της επιλογής του με το σωστό δείκτη (AnswerIndex) της ερώτησης. Αν η απάντηση είναι σωστή, ο ήχος επιτυχίας αναπαράγεται, η βαθμολογία αυξάνεται και το κουμπί της σωστής απάντησης «αναβοσβήνει» πράσινο με τη βοήθεια της FlashButton. Αν η απάντηση είναι λανθασμένη, αναπαράγεται ήχος αποτυχίας, το κουμπί της λάθος απάντησης χρωματίζεται κόκκινο, και η σωστή απάντηση επισημαίνεται με πράσινο. Μετά από μικρή καθυστέρηση, προβάλλεται η επόμενη ερώτηση.

Παράλληλα, κάθε ερώτηση συνοδεύεται από χρονικό όριο, το οποίο καθορίζεται από τη ρύθμιση GameSettings.TimePerQuestionInSeconds. Η μέθοδος StartTimer ξεκινά έναν χρονόμετρο που μετρά αντίστροφα κάθε δευτερόλεπτο μέσω του System.Timers.Timer. Αν ο χρόνος μηδενιστεί πριν απαντήσει ο χρήστης, τότε η τρέχουσα ερώτηση θεωρείται ληγμένη και η επόμενη εμφανίζεται αυτόματα. Η διαχείριση του χρονισμού γίνεται στις μεθόδους OnTimerTick και StopTimer.

Η ShowNextQuestion ελέγχει αν έχουν τελειώσει όλες οι ερωτήσεις και, αν ναι, μεταφέρει τον χρήστη στη σελίδα αποτελεσμάτων ResultsPage, περνώντας την κατηγορία, τη βαθμολογία και το πλήθος των ερωτήσεων. Διαφορετικά, εμφανίζει την επόμενη ερώτηση.

Τέλος, η μέθοδος `LoadQuestionsByCategoryAsync` είναι υπεύθυνη για την ανάγνωση των ερωτήσεων από τα αντίστοιχα αρχεία JSON.


```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="QuizGame.Views.QuizPage"
              BackgroundColor="Transparent"
              Title="Quiz">

    <Grid>
        <!-- Background image -->
        <Image Source="quizbackgroundimage.png"
              Aspect="AspectFill"/>

        <!-- Main content -->
        <VerticalStackLayout Padding="20" Spacing="20"
                          AbsoluteLayout.LayoutFlags="All"
                          AbsoluteLayout.LayoutBounds="0, 0, 1, 1">

            <!-- Timer -->
            <Label x:Name="TimerLabel"
                  Text="Time: 10"
                  FontSize="18"
                  HorizontalOptions="End"/>

            <!-- Question text centered -->
            <Label x:Name="QuestionLabel"
                  Text="Question will appear here"
                  FontSize="22"
                  FontAttributes="Bold"
                  HorizontalOptions="Center"
                  HorizontalTextAlignment="Center"
                  Margin="0,10"/>

            <!-- Answer options container -->
            <VerticalStackLayout x:Name="OptionsContainer"
                              Spacing="10">
                <!-- Answer buttons will be created dynamically -->
            </VerticalStackLayout>

            <!-- Score -->
            <Label x:Name="ScoreLabel"
                  Text="Score: 0"
                  FontSize="16"
                  HorizontalOptions="Center"
                  Margin="0,20,0,0"/>

        </VerticalStackLayout>
    </Grid>
</ContentPage>
```



```

using System.Timers;
using System.Text.Json;
using QuizGame.Models;
using QuizGame.Helpers;
using Microsoft.Maui.Graphics.Text;
using Plugin.Maui.Audio;

namespace QuizGame.Views;

16 references
public partial class QuizPage : ContentPage
{
    private string _category;
    private List<Question> _questions;
    private int _currentQuestionIndex = 0;
    private int _score = 0;
    private int _timeLeft = GameSettings.TimePerQuestionInSeconds;
    private System.Timers.Timer _timer;
    private readonly IAudioManager _audioManager = AudioManager.Current;
    private IAudioPlayer _correctSoundPlayer;
    private IAudioPlayer _wrongSoundPlayer;

    2 references
    public QuizPage(string category)
    {
        InitializeComponent();
        _category = category;
        LoadCategoryQuestions();
        LoadSoundsAsync();
    }

    // LOAD CATEGORY QUESTIONS
    1 reference
    private async void LoadCategoryQuestions()
    {
        _questions = await LoadQuestionsByCategoryAsync(_category);
        ShowNextQuestion();
    }

    // SOUND FOR ANSWERS
    1 reference
    private async Task LoadSoundsAsync()
    {
        var correctStream = await FileSystem.OpenAppPackageFileAsync("correct.mp3");
        _correctSoundPlayer = _audioManager.CreatePlayer(correctStream);

        var wrongStream = await FileSystem.OpenAppPackageFileAsync("wrong.mp3");
        _wrongSoundPlayer = _audioManager.CreatePlayer(wrongStream);
    }
}

```

```

// NEXT QUESTION
3 references
private async void ShowNextQuestion()
{
    if (_currentQuestionIndex >= _questions.Count)
    {
        await Navigation.PushAsync(new ResultsPage(_category, _score, _questions.Count));
        return;
    }

    var question = _questions[_currentQuestionIndex];
    QuestionLabel.Text = question.Text;
    OptionsContainer.Children.Clear();

    for (int i = 0; i < question.Options.Count; i++)
    {
        var button = new Button
        {
            Text = question.Options[i],
            CommandParameter = i,
            BackgroundColor = Colors.Black,
            TextColor = Colors.White
        };
        button.Clicked += OnOptionClicked;
        OptionsContainer.Children.Add(button);
    }

    ScoreLabel.Text = $"Score: {_score}";
    StartTimer();
}

// ANSWER
1 reference
private async void OnOptionClicked(object sender, EventArgs e)
{
    StopTimer();

    var button = (Button)sender;
    int selected = (int)button.CommandParameter;
    int correct = _questions[_currentQuestionIndex].AnswerIndex;

    // BUTTON COLOR
    if (selected == correct)
    {
        _correctSoundPlayer?.Play();
        await FlashButton(button, Colors.Green, Colors.Black, 2); //ANIMATION
        _score++;
    }
    else
    {
        button.BackgroundColor = Colors.Red; // WRONG ANSWER
        button.TextColor = Colors.White;
        _wrongSoundPlayer?.Play();

        // SHOW CORRECT ANSWER
        foreach (var child in OptionsContainer.Children)
        {
            if (child is Button b && (int)b.CommandParameter == correct)
            {
                b.BackgroundColor = Colors.Green;
                b.TextColor = Colors.White;
                break;
            }
        }
    }

    await Task.Delay(1000); // DELAY COLOR DISPLAY
    _currentQuestionIndex++;
    ShowNextQuestion();
}

```

```

// GRAPHICS
1 reference
private async Task FlashButton(Button button, Color color1, Color color2, int flashes)
{
    for (int i = 0; i < flashes; i++)
    {
        button.BackgroundColor = color1;
        await Task.Delay(200);
        button.BackgroundColor = color2;
        await Task.Delay(200);
    }

    // End on the original color
    button.BackgroundColor = color1;
}

// TIMER START
1 reference
private void StartTimer()
{
    _timeLeft = GameSettings.TimePerQuestionInSeconds;
    TimerLabel.Text = $"Time: {_timeLeft}";

    _timer = new System.Timers.Timer(1000);
    _timer.Elapsed += OnTimerTick;
    _timer.Start();
}

// TIMER TICK
1 reference
private void OnTimerTick(object sender, ElapsedEventArgs e)
{
    _timeLeft--;

    MainThread.BeginInvokeOnMainThread(() =>
    {
        TimerLabel.Text = $"Time: {_timeLeft}";
    });

    if (_timeLeft <= 0)
    {
        _timer.Stop();

        MainThread.BeginInvokeOnMainThread(() =>
        {
            _currentQuestionIndex++;
            ShowNextQuestion();
        });
    }
}

// TIMER STOP
1 reference
private void StopTimer()
{
    if (_timer != null)
    {
        _timer.Stop();
        _timer.Dispose();
    }
}

// LOAD QUESTIONS FROM JSON FILES
1 reference
private async Task<List<Question>> LoadQuestionsByCategoryAsync(string category)
{
    string filename = $"Data/{category.ToLower()}.json";
    using var stream = await FileSystem.OpenAppPackageFileAsync(filename);
    using var reader = new StreamReader(stream);
    var json = await reader.ReadToEndAsync();
    var questions = JsonSerializer.Deserialize<List<Question>>(json);
    return questions;
}

```

4.5 Αποτελέσματα

Η σελίδα ResultsPage εμφανίζεται στον χρήστη μόλις ολοκληρώσει ένα quiz. Η σελίδα αυτή είναι υπεύθυνη για την εμφάνιση της βαθμολογίας, της ακρίβειας (ποσοστό σωστών απαντήσεων), την πιθανή ενημέρωση του προσωπικού highscore του χρήστη, καθώς και την προβολή του leaderboard για την κατηγορία που μόλις έπαιξε.

Στον constructor της κλάσης δίνονται ως ορίσματα η κατηγορία του quiz, το σκορ του χρήστη και ο συνολικός αριθμός ερωτήσεων. Οι τιμές αυτές αποθηκεύονται σε ιδιωτικές μεταβλητές και εμφανίζονται στην οθόνη με δύο ετικέτες:

- ScoreLabel, που δείχνει το σκορ
- PercentageLabel, που υπολογίζει την ακρίβεια σε ποσοστό με βάση το σκορ

Μέσα στον constructor καλούνται δύο ασύγχρονες μέθοδοι:

- UpdateHighscoreIfNeededAsync(), που ελέγχει αν ο χρήστης είναι συνδεδεμένος και αν ναι, καλεί την ApiService για να ενημερώσει το highscore του χρήστη για τη συγκεκριμένη κατηγορία, εφόσον το νέο σκορ είναι καλύτερο. Η σύνδεση γίνεται με χρήση του SecureStorage για την ανάκτηση του αποθηκευμένου username.
- LoadLeaderboardAsync(), που καλεί το backend μέσω ApiService για να λάβει τα highscore άλλων παικτών για την ίδια κατηγορία. Αφού παραλάβει τη λίστα των χρηστών και των σκορ τους, τους ταξινομεί κατά σειρά και τους εμφανίζει στο LeaderboardView, μαζί με τη θέση (Rank) κάθε παίκτη.

Εκτός από την προβολή αποτελεσμάτων, η σελίδα περιλαμβάνει δύο επιλογές για τον χρήστη:

- Με το κουμπί "Retry" και τη μέθοδο OnRetryClicked, ο χρήστης μπορεί να ξαναπαιξει αμέσως την ίδια κατηγορία. Η σελίδα QuizPage επανεκκινείται με την ίδια παράμετρο κατηγορίας.
- Με το κουμπί "Main Menu" και τη μέθοδο OnMainMenuClicked, ο χρήστης επιστρέφει στο αρχικό μενού της εφαρμογής (MainMenuPage) μέσω της μεθόδου PopToRootAsync().

```
namespace QuizGame.Views;

15 references
public partial class ResultsPage : ContentPage
{
    private string _category;
    private int _score;
    private int _total;

    1 reference
    public ResultsPage(string category, int score, int total)
    {
        InitializeComponent();

        _category = category;
        _score = score;
        _total = total;

        ScoreLabel.Text = $"Score: {_score}/{_total}";
        PercentageLabel.Text = $"Accuracy: {(int)((double)_score / _total * 100)}%";

        _ = UpdateHighscoreIfNeededAsync();
        _ = LoadLeaderboardAsync();
    }

    // HIGHSCORE UPDATE
    1 reference
    private async Task UpdateHighscoreIfNeededAsync()
    {
        try
        {
            var username = await SecureStorage.GetAsync("username");
            if (!string.IsNullOrEmpty(username))
            {
                var api = new ApiService();
                await api.UpdateCategoryHighscoreAsync(username, _category, _score);
            }
            else
            {
                Console.WriteLine("Username not found in secure storage.");
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error updating highscore: {ex.Message}");
        }
    }
}
```

```
// LEADERBOARD
1 reference
private async Task LoadLeaderboardAsync()
{
    try
    {
        var api = new ApiService();
        var leaderboard = await api.GetLeaderboardAsync(_category);

        // Rank
        var rankedLeaderboard = leaderboard
            .Select((entry, index) => new LeaderboardEntry
            {
                Rank = index + 1,
                Username = entry.Username,
                Highscore = entry.Highscore
            })
            .ToList();

        LeaderboardView.ItemsSource = rankedLeaderboard;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error loading leaderboard: {ex.Message}");
    }
}

// REPLAY QUIZ
0 references
private async void OnRetryClicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new QuizPage(_category));
}

// BACK TO MAIN MENU
0 references
private async void OnMainMenuClicked(object sender, EventArgs e)
{
    await Navigation.PopToRootAsync();
}
```



```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
x:Class="QuizGame.Views.ResultsPage"
Title="Results">

    <Grid>
        <!-- Background image -->
        <Image Source="quizbackgroundimage.png"
Aspect="AspectFill" />

        <!-- Scrollable content -->
        <ScrollView>
            <VerticalStackLayout Padding="30"
Spacing="25"
HorizontalOptions="Center"
VerticalOptions="Center">

                <!-- Score & Accuracy -->
                <Label x:Name="ScoreLabel" FontSize="32" TextColor="Black" />
                <Label x:Name="PercentageLabel" FontSize="24" TextColor="Black" />

                <!-- Buttons -->
                <Button Text="Retry Quiz" Clicked="OnRetryClicked" />
                <Button Text="Return to Main Menu" Clicked="OnMainMenuClicked" />

                <!-- Leaderboard Title -->
                <Label Text="Leaderboard"
FontSize="28"
TextColor="Black"
HorizontalOptions="Center"
Margin="0,10" />

                <!-- Leaderboard Container -->
                <Frame BackgroundColor="Yellow"
CornerRadius="12"
HasShadow="True"
Padding="0"
WidthRequest="320"
HorizontalOptions="Center">

                    <CollectionView x:Name="LeaderboardView"
HeightRequest="320"
SelectionMode="None"
ItemsSource="{Binding}">

                        <!-- Header -->
                        <CollectionView.Header>
                            <Grid Padding="10"
BackgroundColor="#E0E0E0"
ColumnDefinitions="40,*,60">
                                <Label Text="#" FontAttributes="Bold" />
                                <Label Text="Username" Grid.Column="1" FontAttributes="Bold" />
                                <Label Text="Score" Grid.Column="2" FontAttributes="Bold"
HorizontalTextAlignment="End" />
                            </Grid>
                        </CollectionView.Header>

                        <!-- Items -->
                        <CollectionView.ItemTemplate>
                            <DataTemplate>
                                <Grid Padding="10"
ColumnDefinitions="40,*,60"
RowSpacing="8">
                                    <Label Text="{Binding Rank}" TextColor="Black" />
                                    <Label Text="{Binding Username}" TextColor="Black" Grid.Column="1" />
                                    <Label Text="{Binding Highscore}" TextColor="Black" Grid.Column="2"
HorizontalTextAlignment="End" />
                                </Grid>
                            </DataTemplate>
                        </CollectionView.ItemTemplate>
                    </CollectionView>
                </Frame>

            </VerticalStackLayout>
        </ScrollView>
    </Grid>

```

4.6 Ρυθμίσεις

Η σελίδα `SettingsPage` δίνει στον χρήστη τη δυνατότητα να ρυθμίσει δύο βασικές παραμέτρους του παιχνιδιού: την ένταση ήχου και τη δυσκολία. Η τιμή του `VolumeSlider` αρχικοποιείται στο 50, θέτοντας την ένταση ήχου στο 50% ως προεπιλογή.

Η μέθοδος `OnVolumeChanged` ενεργοποιείται κάθε φορά που ο χρήστης μετακινεί τον slider της έντασης. Η νέα τιμή που επιλέχθηκε μετατρέπεται σε ακέραιο και εμφανίζεται δυναμικά στην ετικέτα `VolumeLabel`.

Η μέθοδος `OnDifficultyChecked` ενεργοποιείται όταν ο χρήστης επιλέξει διαφορετική βαθμίδα δυσκολίας μέσω ενός `RadioButton`. Ελέγχεται πρώτα αν η επιλογή ενεργοποιήθηκε ώστε να αγνοούνται τα ανεπιθύμητα συμβάντα κατά την αποεπιλογή άλλων επιλογών. Η τιμή του επιλεγμένου `RadioButton` (π.χ. "Easy", "Medium", "Hard") αποθηκεύεται στη μεταβλητή `difficulty`, και στη συνέχεια ορίζεται ο χρόνος απάντησης ανά ερώτηση στο `GameSettings.TimePerQuestionInSeconds`, με βάση τη δυσκολία:

- 30 δευτερόλεπτα για "Easy"
- 20 δευτερόλεπτα για "Medium"
- 10 δευτερόλεπτα για "Hard"

Μετά την επιλογή, η τιμή δυσκολίας αποθηκεύεται στις ρυθμίσεις της συσκευής μέσω της `Preferences.Set("Difficulty", difficulty)`, επιτρέποντας την ανάκτησή της σε επόμενη εκκίνηση της εφαρμογής.


```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="QuizGame.Views.SettingsPage"
              Title="Settings">

    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="30">

            <Label Text="🔊 Audio Settings" FontSize="24" />

            <Slider x:Name="VolumeSlider"
                    Minimum="0"
                    Maximum="100"
                    Value="50"
                    ValueChanged="OnVolumeChanged"/>
            <Label x:Name="VolumeLabel" Text="Volume: 50%" FontSize="16" />

            <Label Text="Difficulty Settings" FontSize="24" />

            <RadioButton x:Name="EasyRadio" Content="Easy" GroupName="Difficulty" CheckedChanged="OnDifficultyChecked" />
            <RadioButton x:Name="MediumRadio" Content="Medium" GroupName="Difficulty" CheckedChanged="OnDifficultyChecked" />
            <RadioButton x:Name="HardRadio" Content="Hard" GroupName="Difficulty" CheckedChanged="OnDifficultyChecked" />

        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

```
using QuizGame.Helpers;
namespace QuizGame.Views;

15 references
public partial class SettingsPage : ContentPage
{
    1 reference
    public SettingsPage()
    {
        InitializeComponent();
        VolumeSlider.Value = 50;
    }

    // VOLUME
    0 references
    private void OnVolumeChanged(object sender, ValueChangedEventArgs e)
    {
        int volume = (int)e.NewValue;
        VolumeLabel.Text = $"Volume: {volume}%";
    }

    // DIFFICULTY
    0 references
    private void OnDifficultyChecked(object sender, CheckedChangedEventArgs e)
    {
        if (!e.Value)
            return;

        var selected = (RadioButton)sender;
        string difficulty = selected.Content.ToString();
        Console.WriteLine($"Difficulty selected: {difficulty}");

        switch (difficulty)
        {
            case "Easy":
                GameSettings.TimePerQuestionInSeconds = 30;
                break;
            case "Medium":
                GameSettings.TimePerQuestionInSeconds = 20;
                break;
            case "Hard":
                GameSettings.TimePerQuestionInSeconds = 10;
                break;
        }

        // SAVE PREFERENCE
        Preferences.Set("Difficulty", difficulty);
    }
}
```

4.7 API Service

Η κλάση ApiService, αποτελεί τον ενδιάμεσο κρίκο (client-side service) ανάμεσα στην εφαρμογή QuizGame και το backend API. Μέσω αυτής της κλάσης, η εφαρμογή μπορεί να στέλνει αιτήματα HTTP για ενέργειες όπως εγγραφή χρήστη, σύνδεση, ενημέρωση highscore και ανάκτηση leaderboard. Ο κώδικας χρησιμοποιεί το HttpClient και λειτουργεί ασύγχρονα για να μην μπλοκάρει την κύρια διεπαφή του χρήστη κατά την επικοινωνία με τον server. Η μέθοδος ApiService() είναι ο κατασκευαστής της υπηρεσίας. Δημιουργεί ένα νέο HttpClient και του ορίζει ως BaseAddress το http://localhost:5165/, δηλαδή τη διεύθυνση όπου «τρέχει» το backend API της εφαρμογής κατά την ανάπτυξη. Όλες οι κλήσεις προς το API θα βασίζονται σε αυτήν τη διεύθυνση. Η μέθοδος LoginAsync, υλοποιεί τη σύνδεση χρήστη. Στέλνει ένα POST αίτημα με το username και password στη διαδρομή api/users/login. Αν η σύνδεση είναι επιτυχής, τότε διαβάζει το JSON περιεχόμενο της απάντησης και το μετατρέπει σε αντικείμενο τύπου LoginResult, το οποίο περιέχει το id και το username του χρήστη. Η μέθοδος επιστρέφει true αν όλα πάνε καλά και false αν αποτύχει. Η μέθοδος UpdateCategoryHighscoreAsync, στέλνει στο backend το νέο σκορ ενός χρήστη για μία συγκεκριμένη κατηγορία. Το αίτημα περιέχει το username, την category και το score, και αποστέλλεται με POST στη διαδρομή api/users/update-category-highscore.

Ο server μπορεί να ενημερώσει τη βάση δεδομένων μόνο αν το νέο σκορ είναι καλύτερο από το προηγούμενο. Η μέθοδος επιστρέφει true σε περίπτωση επιτυχίας, αλλιώς false. Η μέθοδος GetLeaderboardAsync είναι υπεύθυνη για την ανάκτηση του πίνακα κατάταξης (leaderboard) μιας συγκεκριμένης κατηγορίας. Στέλνει GET αίτημα στη διαδρομή api/users/leaderboard/{category} και λαμβάνει πίσω μια λίστα από LeaderboardEntry αντικείμενα. Οι δύο βοηθητικές κλάσεις περιγράφουν τη δομή των δεδομένων που λαμβάνονται από το API. Η LoginResult: περιέχει id και username και χρησιμοποιείται κατά την επιτυχημένη σύνδεση. Τέλος, η LeaderboardEntry περιέχει το Rank, Username, και Highscore ενός χρήστη και χρησιμοποιείται για την εμφάνιση της λίστας κατάταξης στην ResultsPage. Όλες οι μέθοδοι είναι ασύγχρονες, διατηρώντας ομαλή και άμεση την εμπειρία χρήσης της εφαρμογής. Έτσι, η εφαρμογή QuizGame μπορεί να προσφέρει δυναμική αλληλεπίδραση με απομακρυσμένο διακομιστή χωρίς καθυστερήσεις ή παγώματα στην οθόνη του χρήστη.

```
1 reference
public class LoginResult
{
    0 references
    public int id { get; set; }
    0 references
    public string username { get; set; }
}

5 references
public class LeaderboardEntry
{
    1 reference
    public int Rank { get; set; }
    2 references
    public string Username { get; set; }
    2 references
    public int Highscore { get; set; }
}
```

```

5 references
public class ApiService
{
    private readonly HttpClient _httpClient;

    4 references
    public ApiService()
    {
        _httpClient = new HttpClient();
        _httpClient.BaseAddress = new Uri("http://localhost:5165/");
    }

    //REGISTER
    1 reference
    public async Task<bool> RegisterAsync(string username, string password)
    {
        var registerData = new { username, password };
        var response = await _httpClient.PostAsJsonAsync("api/users/register", registerData);
        return response.IsSuccessStatusCode;
    }

    //LOGIN
    1 reference
    public async Task<bool> LoginAsync(string username, string password)
    {
        var loginData = new { username, password };
        var response = await _httpClient.PostAsJsonAsync("api/users/login", loginData);

        if (response.IsSuccessStatusCode)
        {
            var result = await response.Content.ReadFromJsonAsync<LoginResult>();
            // Optionally save user id or token
            return true;
        }

        return false;
    }

    //CATEGORY HIGHSCORE
    1 reference
    public async Task<bool> UpdateCategoryHighscoreAsync(string username, string category, int score)
    {
        var request = new
        {
            Username = username,
            Category = category,
            Score = score
        };

        var response = await _httpClient.PostAsJsonAsync("api/users/update-category-highscore", request);
        return response.IsSuccessStatusCode;
    }

    //LEADERBOARD
    1 reference
    public async Task<List<LeaderboardEntry>> GetLeaderboardAsync(string category)
    {
        var response = await _httpClient.GetAsync($"api/users/leaderboard/{category}");
        if (!response.IsSuccessStatusCode) return new List<LeaderboardEntry>();

        var result = await response.Content.ReadFromJsonAsync<List<LeaderboardEntry>>();
        return result ?? new List<LeaderboardEntry>();
    }
}

```

4.8 Audio Service

Η κλάση `BackgroundAudioService`, είναι υπεύθυνη για την αναπαραγωγή μουσικής υπόκρουσης (background music) στην εφαρμογή `QuizGame`. Πρόκειται για μια βοηθητική υπηρεσία που αξιοποιεί το πακέτο `Plugin.Maui.Audio`, ένα πρόσθετο εργαλείο που επιτρέπει την εύκολη αναπαραγωγή ήχου σε εφαρμογές που φτιάχνονται με `.NET MAUI`. Ο constructor της κλάσης `BackgroundAudioService` δέχεται ως όρισμα έναν διαχειριστή ήχου (`IAudioManager`) ο οποίος αποθηκεύεται στην ιδιωτική μεταβλητή `_audioManager`. Αυτός ο manager είναι υπεύθυνος για τη δημιουργία audio players που μπορούν να παίξουν αρχεία ήχου. Με αυτόν τον τρόπο, η υπηρεσία δεν δημιουργεί μόνη της το `IAudioManager`, αλλά τον λαμβάνει από εξωτερικό context, κάτι που διευκολύνει τη χρήση της σε διαφορετικά μέρη της εφαρμογής (π.χ. μέσω dependency injection). Η ασύγχρονη μέθοδος `StartBackgroundMusicAsync` ξεκινά την αναπαραγωγή της μουσικής υπόκρουσης. Η μέθοδος `StopMusic()` σταματά τη μουσική. Αν ο player υπάρχει, καλεί την `.Stop()` για να διακοπεί αμέσως η αναπαραγωγή. Δεν καταστρέφει τον player, αλλά απλώς σταματά την τρέχουσα μουσική.

```
using Plugin.Maui.Audio;

namespace QuizGame.Services
{
    3 references
    public class BackgroundAudioService
    {
        private readonly IAudioManager _audioManager;
        private IAudioPlayer _player;

        0 references
        public BackgroundAudioService(IAudioManager audioManager)
        {
            _audioManager = audioManager;
        }

        1 reference
        public async Task StartBackgroundMusicAsync()
        {
            if (_player != null && _player.IsPlaying)
                return;

            var stream = await FileSystem.OpenAppPackageFileAsync("theme.mp3");
            _player = _audioManager.CreatePlayer(stream);
            _player.Loop = true;
            _player.Play();
        }

        0 references
        public void StopMusic()
        {
            _player?.Stop();
        }
    }
}
```

Κεφάλαιο 5: Αποτελέσματα

Η υλοποίηση της εφαρμογής Quiz Game οδήγησε στην επίτευξη ενός πλήρως λειτουργικού πληροφοριακού συστήματος που υποστηρίζει διαδραστική αλληλεπίδραση χρήστη, αποθήκευση και ανάκτηση δεδομένων από απομακρυσμένο διακομιστή, καθώς και αξιολόγηση επιδόσεων μέσω δυναμικών πινάκων κατάταξης.

5.1 Δεδομένα και Παραδοχές

Η λειτουργία και αξιολόγηση της εφαρμογής βασίστηκε στις εξής βασικές παραδοχές:

- Κάθε χρήστης είναι μοναδικός, αναγνωρίζεται μέσω ονόματος χρήστη (username), και μπορεί να έχει ξεχωριστό υψηλό σκορ (highscore) για κάθε κατηγορία quiz.
- Η αποθήκευση στοιχείων πραγματοποιείται σε βάση δεδομένων MySQL, μέσω backend API
- Τα δεδομένα αποστέλλονται μέσω RESTful HTTP requests και λαμβάνονται σε μορφή JSON.
- Η εφαρμογή υποστηρίζει πολλαπλές κατηγορίες ερωτήσεων και για κάθε κατηγορία καταγράφεται το υψηλότερο σκορ του χρήστη.
- Η πιστοποίηση ταυτότητας γίνεται μέσω μηχανισμού login, με χρήση hashing για την ασφαλή αποθήκευση του κωδικού

5.2 Παρουσίαση Αποτελεσμάτων

Τα βασικά αποτελέσματα της ανάπτυξης περιλαμβάνουν:

- Επιτυχής λειτουργία backend API με endpoints για εγγραφή, σύνδεση και ενημέρωση σκορ. Τα API ελέγχθηκαν και ανταποκρίνονται σωστά, με κατάλληλους ελέγχους σφαλμάτων και μηνύματα απόκρισης.
- Αποδοτική επικοινωνία μεταξύ frontend και backend, μέσω της κλάσης ApiService, η οποία καθορίζει τις κλήσεις προς τον διακομιστή.
- Ορθή καταγραφή σκορ ανά κατηγορία, με αποθήκευση μόνο του υψηλότερου σκορ ανά χρήστη, γεγονός που περιορίζει τον όγκο των δεδομένων και επιτρέπει εύκολη ανάκτηση για την εμφάνιση των leaderboards.
- Δυναμική ενημέρωση leaderboard με δυνατότητα ταξινόμησης παικτών βάσει σκορ για κάθε κατηγορία.

5.3 Σχολιασμός Αποτελεσμάτων

Η εφαρμογή λειτουργεί ομαλά σε συνθήκες πραγματικής χρήσης και προσφέρει μια φιλική εμπειρία στον τελικό χρήστη. Η σωστή χρήση του REST API, σε συνδυασμό με την ασφαλή αποθήκευση και την ξεκάθαρη αρχιτεκτονική των δεδομένων (Users ↔ Highscores), επιβεβαιώνει την επιτυχή ενοποίηση του backend και του frontend.

Η απόφαση να καταγράφεται μόνο το υψηλότερο σκορ αντί για πλήρες ιστορικό κάθε παιχνιδιού, απλοποίησε σημαντικά τον σχεδιασμό της βάσης και περιορίσε τον απαιτούμενο αποθηκευτικό χώρο, χωρίς να μειώνει τη λειτουργικότητα της εφαρμογής.

Η δομή της βάσης, σε συνδυασμό με τη modular λογική του κώδικα, καθιστά εύκολη την επεκτασιμότητα της εφαρμογής (π.χ. προσθήκη νέων κατηγοριών, πολλαπλών γλωσσών, κ.λπ.).

Κεφάλαιο 6: Προκλήσεις & Προβλήματα

Κατά την υλοποίηση της εφαρμογής παρουσιάστηκαν διάφορες τεχνικές και σχεδιαστικές προκλήσεις, οι οποίες αντιμετωπίστηκαν μέσω μελέτης, τεκμηρίωσης, δοκιμών και συνεχούς βελτίωσης του κώδικα.

6.1 Διαχείριση Επικοινωνίας Frontend – Backend

Η ανάπτυξη της επικοινωνίας μεταξύ .NET MAUI (frontend) και ASP.NET Core Web API (backend) απαιτούσε προσεκτικό χειρισμό, ιδιαίτερα σε ό,τι αφορά τη σωστή διαχείριση των HTTP αιτημάτων και απαντήσεων, τη μορφοποίηση των JSON δεδομένων για συμβατότητα και τη διαχείριση των exceptions.

6.2 Υλοποίηση Leaderboards και Ενημέρωση Σκορ

Μία από τις βασικότερες προκλήσεις ήταν η σωστή υλοποίηση της λογικής για την αποθήκευση μοναδικού σκορ ανά κατηγορία και χρήστη, ώστε να αποφεύγεται η καταγραφή κάθε απόπειρας. Αρχικά, υπήρχε πρόβλημα με την εγγραφή πολλών εγγραφών για τον ίδιο χρήστη και κατηγορία, το οποίο επιλύθηκε με την προσθήκη κατάλληλης λογικής στο backend.

6.3 Debugging

Ο έλεγχος και η αποσφαλμάτωση της εφαρμογής σε διαφορετικές πλατφόρμες (κυρίως Android και Windows) παρουσίασε διαφορές σε συμπεριφορά και απαιτήσεις δικαιωμάτων, με αποτέλεσμα να απαιτείται επιπλέον παραμετροποίηση.

Κεφάλαιο 7: Προτεινόμενες Βελτιώσεις

Η εφαρμογή που υλοποιήθηκε αποτελεί μια πλήρως λειτουργική βάση για ένα εκπαιδευτικό quiz game. Ωστόσο, εντοπίζονται περιθώρια εξέλιξης, τα οποία θα μπορούσαν να ενισχύσουν περαιτέρω τη χρηστικότητα και την εμπειρία του χρήστη.

7.1 Προσθήκη Ερωτήσεων μέσω Backend Panel

Προς το παρόν οι ερωτήσεις είναι στατικές και αποθηκευμένες τοπικά. Θα ήταν χρήσιμο να δημιουργηθεί ένα διαχειριστικό περιβάλλον (admin panel) για δυναμική εισαγωγή, τροποποίηση και διαγραφή ερωτήσεων από το backend.

7.2 Achievements

Η προσθήκη μιας λειτουργίας επιβραβεύσεων θα πρόσφερε μια σημαντική αίσθηση επιτυχίας και ολοκλήρωσης στον παίκτη, καθώς θα επέστρεφε για να κερδίσει “τρόπαια” που θα αποδίδονταν εκπληρώνοντας κάποια επιτεύγματα κατά την διάρκεια του παιχνιδιού.

7.3 Πολλαπλές Γλώσσες

Η υποστήριξη πολλαπλών γλωσσών θα καθιστούσε την εφαρμογή προσβάσιμη σε ευρύτερο κοινό, προσφέροντας επιλογή γλώσσας κατά την έναρξη.

7.4 Game Modes

Για να αποφευχθεί η αίσθηση της ρουτίνας και η δυσaréσκεια του χρήστη, θα ήταν χρήσιμο να προστεθούν και άλλα game modes (π.χ. βρες την χώρα στον χάρτη) με σκοπό την περαιτέρω εξέλιξη της εφαρμογής αλλά και της ψυχαγωγίας του παίκτη.

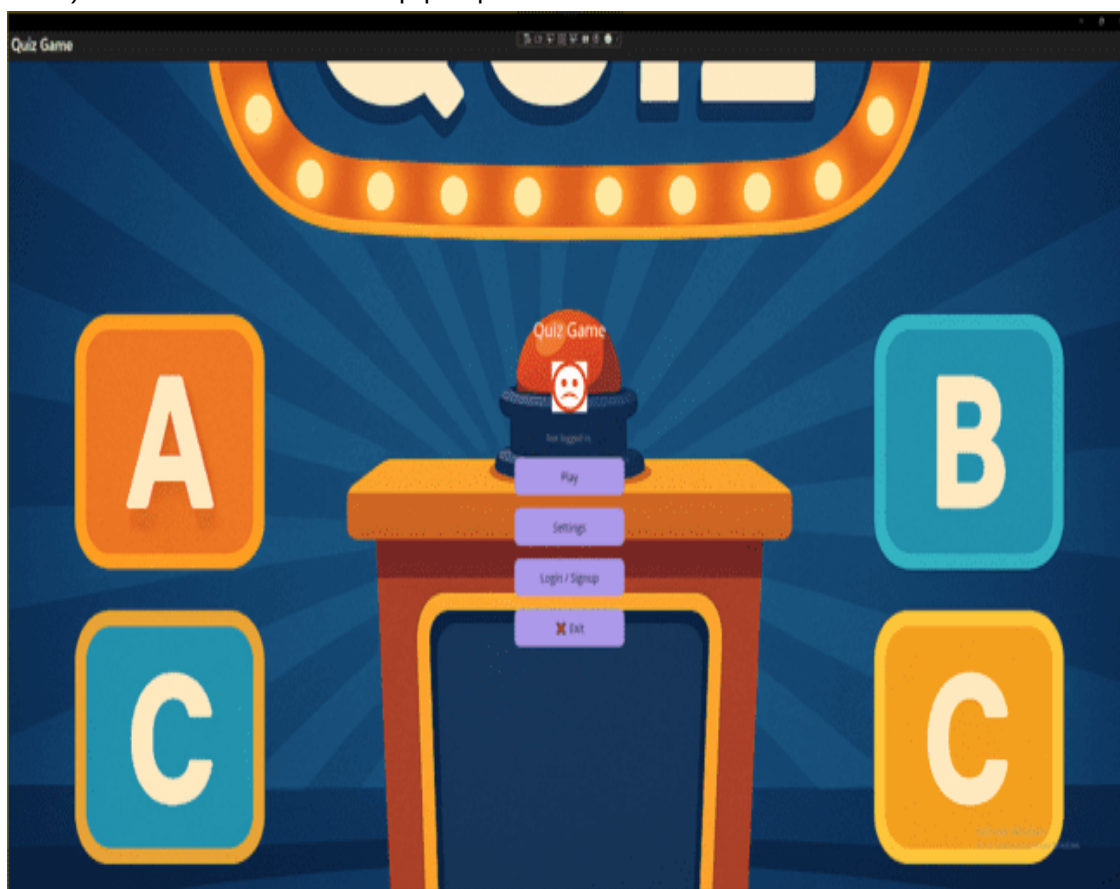
Κεφάλαιο 8: Εγχειρίδιο Χρήστη

Ακολουθούν αναλυτικές οδηγίες για την ορθή χρήση της εφαρμογής.

8.1 Αρχική Σελίδα

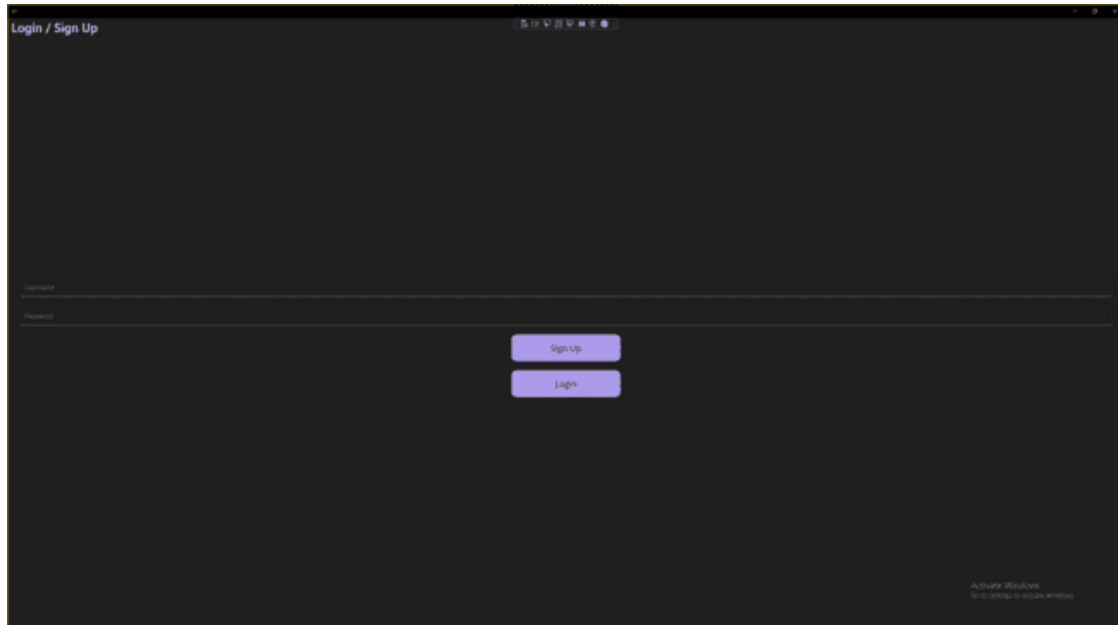
Η αρχική σελίδα της εφαρμογής αποτελεί τον τρόπο πλοήγησης του χρήστη. Μέσα από αυτή, μπορεί να αποκτήσει πρόσβαση σε όλες τις λειτουργίες που διαθέτει το παιχνίδι. Επιπλέον, με την χρήση ενός εικονιδίου, καταλαβαίνει αν είναι συνδεδεμένος ή όχι στον λογαριασμό του. Συγκεκριμένα μπορεί να:

- 1) Παίξει το παιχνίδι
- 2) Πραγματοποιήσει λογαριασμό ή να συνδεθεί σε έναν ήδη υπάρχων
- 3) Προσαρμόσει την εμπειρία του μέσω των ρυθμίσεων
- 4) Κλείσει την εφαρμογή πραγματοποιώντας έξοδο
- 5) Αποσυνδεθεί από τον λογαριασμό του



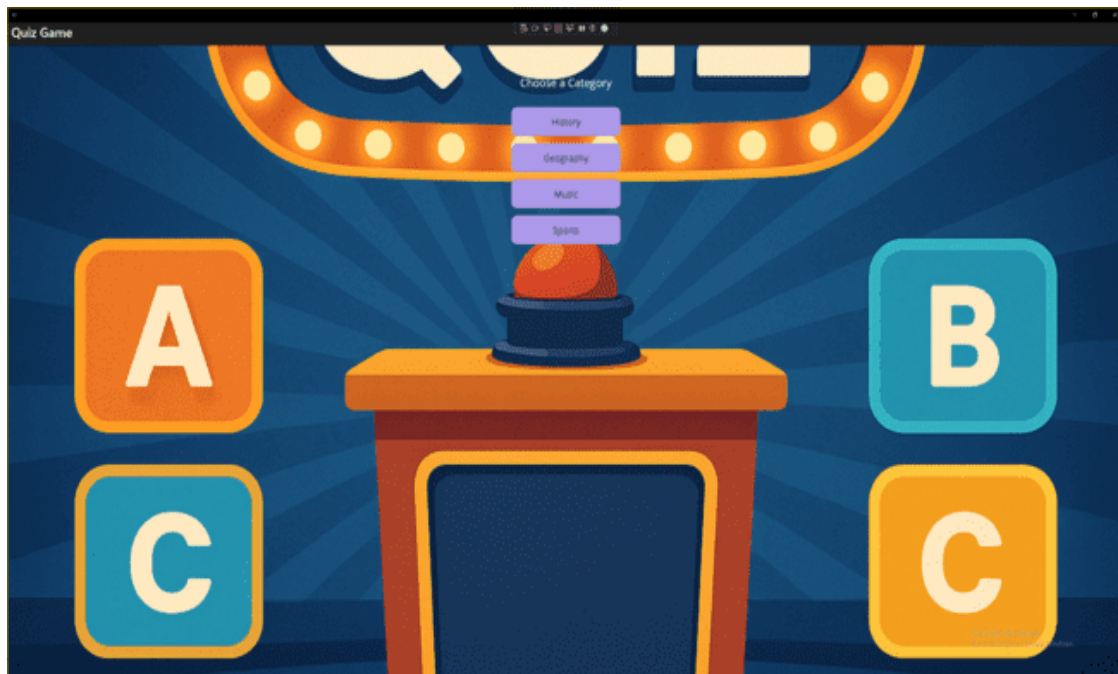
8.2 Σύνδεση/Εγγραφή

Για να εγγραφεί ένας χρήστης στο σύστημα, θα πρέπει να συμπληρώσει τα κενά πεδία «Username» & «Password» και να πατήσει το κουμπί «Sign Up». Το ίδιο ακριβώς κάνει και για την σύνδεση, αλλά πατάει το κουμπί «Login».



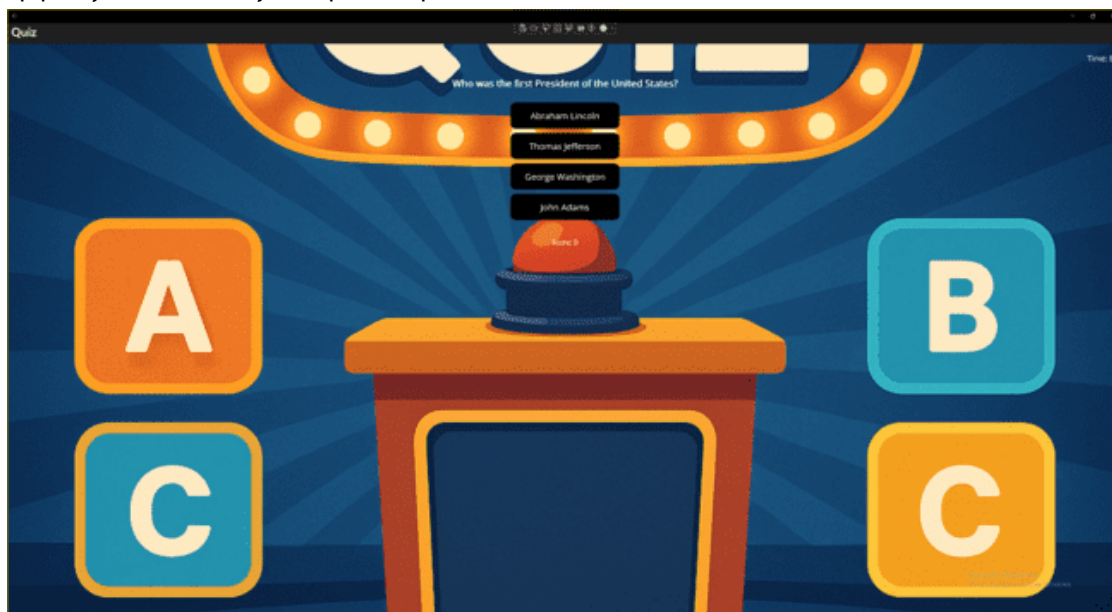
8.3 Επιλογή Κατηγορίας

Σε αυτή την φάση, επιλέγει όποια κατηγορία θέλει πατώντας ένα από τα κουμπιά που υπάρχουν στην οθόνη με τα ονόματα των κατηγοριών.



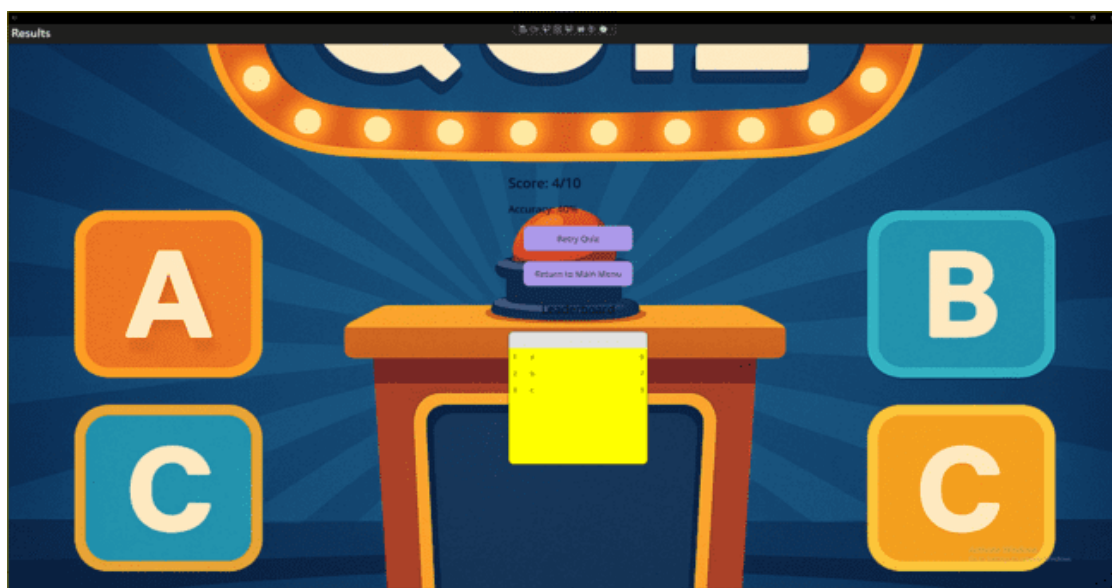
8.4 Απάντηση Ερωτήσεων

Μόλις αρχίσει το παιχνίδι, ο παίκτης βλέπει τις υποψήφιες απαντήσεις στην ερώτηση που του τίθεται και επιλέγει την σωστή πατώντας το αντίστοιχο κουμπί. Αν η απάντηση είναι σωστή, το κουμπί θα αλλάξει χρώμα σε πράσινο και θα έρθει η επόμενη ερώτηση. Αν η απάντηση είναι λάθος, τότε το χρώμα της επιλεγμένης απάντησης γίνεται κόκκινο και εμφανίζεται η σωστή απάντηση με πράσινο χρώμα. Επίσης, ο χρόνος που απομένει για να απαντήσει στην ερώτηση εμφανίζεται πάνω δεξιά στην οθόνη.



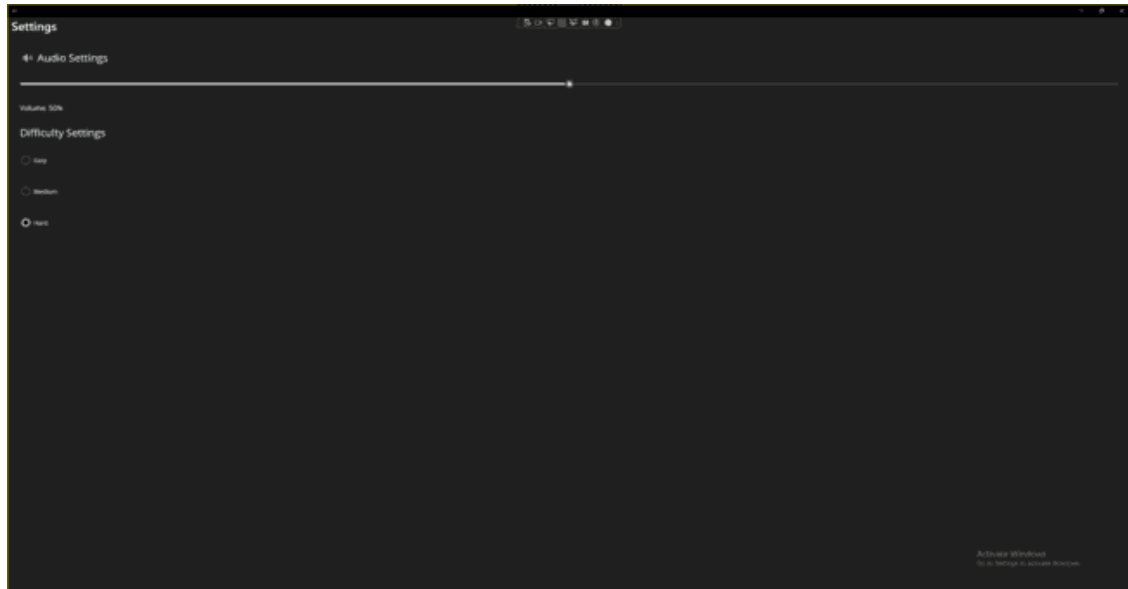
8.5 Αποτελέσματα

Στην τελική οθόνη του παιχνιδιού, ο παίκτης βλέπει τα αποτελέσματα από το Quiz, καθώς και έναν πίνακα με τα υψηλότερα σκορ που έχουν επιτύχει άλλοι παίκτες σε αυτή την κατηγορία. Από εκεί και πέρα, έχει την δυνατότητα να επαναλάβει το Quiz ή να επιστρέψει στην αρχική σελίδα.



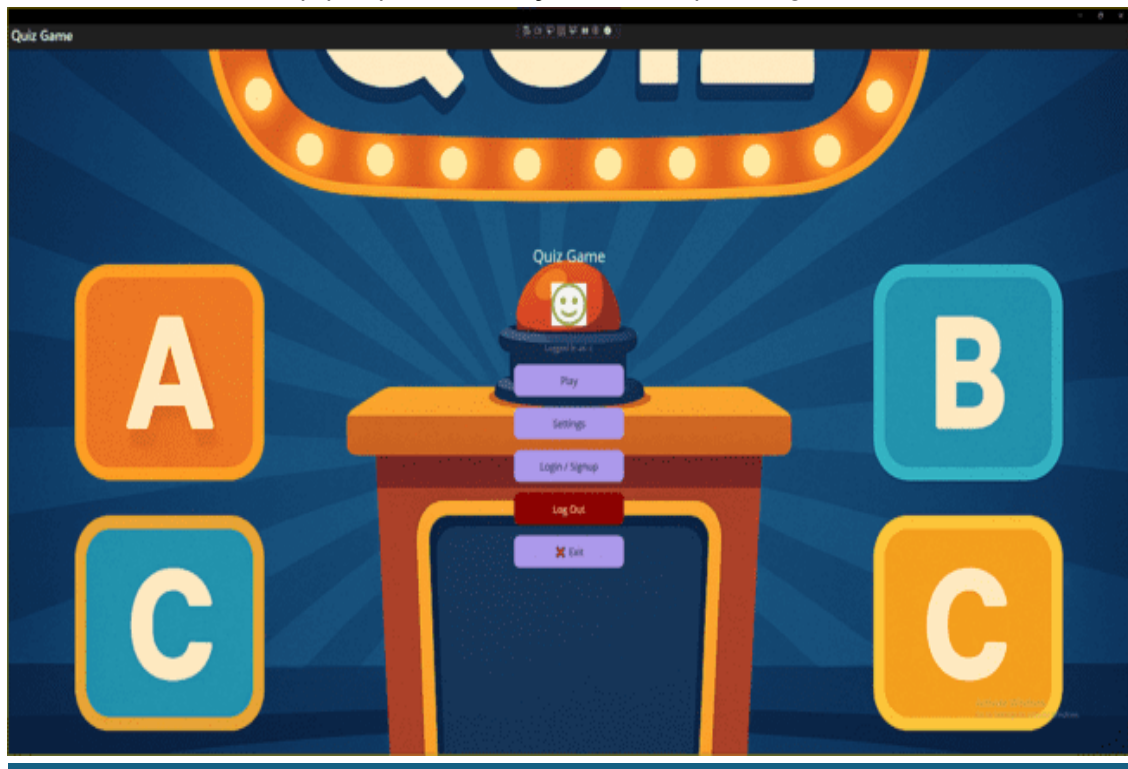
8.6 Ρυθμίσεις

Μέσω των ρυθμίσεων, ο χρήστης έχει την δυνατότητα να αλλάξει την δυσκολία του παιχνιδιού επιλέγοντας ένα από τα κουμπιά «Easy», «Medium» & «Hard», ενώ μπορεί να ρυθμίσει τον ήχο στα επίπεδα που επιθυμεί σέρνοντας τη μπάρα έντασης με το ποντίκι.



8.7 Αποσύνδεση/Εξοδος

Για να πραγματοποιήσει έξοδο από την εφαρμογή, αρκεί να πατήσει το κουμπί «Exit». Για να αποσυνδεθεί από τον λογαριασμό του απλώς πατά το κουμπί «Logout».



Συμπεράσματα

Η υλοποίηση της εφαρμογής επιτεύχθηκε με επιτυχία, επιτρέποντας τη δημιουργία ενός διαδραστικού, φιλικού προς τον χρήστη περιβάλλοντος που συνδυάζει την τεχνολογία με την εκπαιδευτική ψυχαγωγία. Η χρήση του .NET MAUI σε συνδυασμό με το ASP.NET Core Web API και το Entity Framework Core επέτρεψε τη γρήγορη και αποδοτική ανάπτυξη μιας εφαρμογής με σύγχρονες προδιαγραφές και καθαρή αρχιτεκτονική.

Κατά την ανάπτυξη της εφαρμογής, αποκτήθηκαν ουσιαστικές γνώσεις σε τεχνολογίες όπως η πολυπλατφορμική ανάπτυξη, η διαχείριση API, η ασύγχρονη επικοινωνία και η αλληλεπίδραση με σχεσιακές βάσεις δεδομένων. Παράλληλα, καλλιεργήθηκαν δεξιότητες σχετικές με την ανάλυση απαιτήσεων, τον εντοπισμό και επίλυση σφαλμάτων (debugging), καθώς και τη διαχείριση της εμπειρίας χρήστη.

Η εφαρμογή μπορεί να αποτελέσει τη βάση για περαιτέρω ανάπτυξη, σύμφωνα με τις προτάσεις που αναλύθηκαν. Το έργο ανέδειξε τη σημασία της σύνδεσης θεωρητικής γνώσης και πρακτικής εφαρμογής, και προσέφερε πολύτιμη εμπειρία σε πραγματικές συνθήκες ανάπτυξης λογισμικού.

Πίνακας Ορολογίας & Αρκτικόλεξων

Όρος	Επεξήγηση
API	Application Programming Interface – Διασύνδεση μεταξύ διαφορετικών εφαρμογών.
.NET MAUI	.NET Multi-platform App UI – Πλατφόρμα για ανάπτυξη εφαρμογών σε Android, iOS, Windows, macOS.
ASP.NET Core	Πλαίσιο για τη δημιουργία σύγχρονων web εφαρμογών και APIs με .NET.
Entity Framework (EF)	ORM εργαλείο της Microsoft για επικοινωνία εφαρμογών .NET με βάσεις δεδομένων.
ORM	Object-Relational Mapping – Χαρτογράφηση αντικειμένων σε σχεσιακά δεδομένα.
REST / RESTful API	Representational State Transfer – Στυλ αρχιτεκτονικής για web υπηρεσίες.
JSON	JavaScript Object Notation – Μορφή ανταλλαγής δεδομένων μεταξύ frontend και backend.
HTTP	HyperText Transfer Protocol – Πρωτόκολλο επικοινωνίας για το διαδίκτυο.
Frontend	Το οπτικό και λειτουργικό μέρος της εφαρμογής που βλέπει ο χρήστης.
Backend	Το τμήμα της εφαρμογής που χειρίζεται τη λογική και τη διαχείριση δεδομένων.
Leaderboard	Πίνακας κατάταξης που εμφανίζει τους χρήστες με τις υψηλότερες επιδόσεις.
DTO	Data Transfer Object – Αντικείμενο για μεταφορά δεδομένων μεταξύ συστημάτων.
UI / UX	User Interface / User Experience – Διεπαφή και εμπειρία χρήστη αντίστοιχα.
Highscore	Η υψηλότερη βαθμολογία που πέτυχε ένας χρήστης σε ένα quiz.
Gamification	Η ενσωμάτωση μηχανισμών παιχνιδιού (όπως πόντοι, leaderboard κ.ά.) σε μη παιγνιώδεις εφαρμογές για αύξηση εμπλοκής.

Βιβλιογραφία

- Το πέρασμα από την Java στη C#, Ε. Αλέπης & Ι-Χ Παναγιωτόπουλος
- Αντικειμενοστρεφής Τεχνολογία Λογισμικού, Μαρία Κ. Βίρβου
- Σχεσιακές Βάσεις Δεδομένων, Ευάγγελος Κεχρής
- Εισαγωγή στις επιστήμες της Παιδαγωγικής, Ηλίας Γ. Ματσαγγούρας
- Διαδικτυακή Κοινότητα Προγραμματιστών
Stack Overflow: <https://stackoverflow.com/>
- Microsoft Learn – .NET MAUI Documentation <https://learn.microsoft.com/en-us/dotnet/maui/>
- Microsoft Learn – ASP.NET Core Web API <https://learn.microsoft.com/en-us/aspnet/core/web-api/>
- MySQL Documentation Official Website
<https://dev.mysql.com/doc/>
- NET MAUI For Beginners Apress, Gerald Versluis (2022)
<https://doi.org/10.1007/978-1-4842-8224-6>
- Foundations of Game-Based Learning, Plass, J. L., Homer, B. D., & Kinzer, C. K. (2015)
- Εισαγωγή στην Ανάπτυξη Εφαρμογών για Κινητές Συσκευές, Κ. Καρακατσάνης, Εκδόσεις ΤΖΙΟΛΑ (2019)
- Αρχιτεκτονικές και Τεχνολογίες Λογισμικού: REST, APIs, και σύγχρονα frameworks, Ε. Καρύδης, Εκδόσεις Δίσιγμα (2021)
- Tselepatiotis Michail, Efthimios Alepis:
Adaptive Learning in Mobile Serious Games: A Personalized Approach Using AI for General Knowledge Quizzes. [IISA 2024](#): 1-8
- Tselepatiotis Michail, Efthimios Alepis:
Design of Real-Time Multiplayer Word Game for the Android Platform Using Firebase and Fuzzy Logic. [IISA 2023](#): 1-8
- Eugenia A. Politou, Efthimios Alepis, Maria Virvou, Constantinos Patsakis:
Privacy and Data Protection Challenges in the Distributed Era. Springer 2022, ISBN 978-3-030-85442-3, pp. 1-185
- Efthymios Alepis, Aristeia Kontogianni:
Smartphone Crowdsourcing and Data Sharing Towards Advancing User Experience and Mobile Services. [Int. J. Interact. Mob. Technol.](#) **14(3)**: 38-53 (2020)

- Efthimios Alepis, Constantinos Patsakis^{ID}:
Unravelling Security Issues of Runtime Permissions in Android. [J. Hardw. Syst. Secur. 3\(1\)](#): 45-63 (2019)
- Andreas Karavokyris, Efthimios Alepis:
Software Measures for Common Design Patterns Using Visual Studio Code Metrics. [IISA 2018](#): 1-7
- Vasiliki Matzavela, Efthimios Alepis:
A survey for the evolution of adaptive learning in mobile and electronic devices. [IISA 2017](#): 1-5
- Efthimios Alepis, Constantinos Patsakis:
Trapped by the UI: The Android Case. [RAID 2017](#): 334-354
- Efthimios Alepis, Constantinos Patsakis:
The All Seeing Eye: Web to App Intercommunication for Session Fingerprinting in Android. [SpaCCS 2017](#): 93-107
- Efthimios Alepis, Maria Virvou:
Object-Oriented User Interfaces for Personalized Mobile Learning. [Intelligent Systems Reference Library](#) 64, Springer 2014, ISBN 978-3-642-53850-6, pp. 1-129
- Efthimios Alepis, Maria Virvou, Savas Drakoulis:
Human smartphone interaction: Exploring smartphone senses. [IISA 2014](#): 44-48
- Efthymios Alepis, Maria Virvou:
Multimodal object oriented user interfaces in mobile affective interaction. [Multim. Tools Appl. 59\(1\)](#): 41-63 (2012)
- Efthymios Alepis, Maria Virvou:
Object oriented architecture for affective multimodal e-learning interfaces. [Intell. Decis. Technol. 4\(3\)](#): 171-180 (2010)
- Vasiliki Matzavela, Efthimios Alepis:
An application of self-assessment of students in mathematics with intelligent decision systems: questionnaire, design and implementation at digital education. [Educ. Inf. Technol. 28\(11\)](#): 15365-15380 (2023)
- Efthymios Alepis, Maria Virvou, Katerina Kabassi:
Mobile Education: Towards Affective Bi-modal Interaction for Adaptivity. [Int. J. Interact. Mob. Technol. 3\(2\)](#): 40-45 (2009)